

# Accuracy

---

## 1. Overview

Accuracy is the fraction of predictions a classification model gets correct. It is one of the simplest and most commonly reported evaluation metrics.

It is useful when:

- Classes are balanced.
- All types of mistakes have similar cost.
- You need a quick high-level model comparison.

Real-world usage:

- Image classification with balanced labels.
- Spam vs non-spam classification when both classes are equally important.
- Benchmark leaderboards where datasets are curated and balanced.

Accuracy is not enough when classes are imbalanced. A fraud model can get 99% accuracy by predicting "not fraud" for every transaction if only 1% of transactions are fraud.

## 2. Intuition

Accuracy asks: "Out of all answers, how many were correct?"

Example:

If a model predicts 90 labels correctly out of 100 examples, accuracy is 90%.

Analogy: In an exam with 100 multiple-choice questions, accuracy is the percentage of questions answered correctly.

## 3. Prerequisites

- Classification basics
- True positive, true negative, false positive, false negative
- Confusion matrix
- Train/validation/test split
- Class imbalance
- Basic probability and ratios

## 4. Core Concepts

| Subtopic            | Meaning                                  | Why it matters               | Simple example       | Interview angle               |
|---------------------|------------------------------------------|------------------------------|----------------------|-------------------------------|
| Correct predictions | Predictions where <code>y_pred ==</code> | Directly determines accuracy | Cat predicted as cat | "How is accuracy calculated?" |

| Subtopic                     | Meaning                                  | Why it matters                                      | Simple example                     | Interview angle                                     |
|------------------------------|------------------------------------------|-----------------------------------------------------|------------------------------------|-----------------------------------------------------|
| <b><math>y_{true}</math></b> |                                          |                                                     |                                    |                                                     |
| Total samples                | Number of evaluated examples             | Denominator of metric                               | 100 test images                    | "Does dataset size affect reliability?"             |
| Class balance                | Whether classes have similar frequencies | Accuracy is reliable only when classes are balanced | 50 cats, 50 dogs                   | "Why is accuracy misleading on imbalanced data?"    |
| Error rate                   | Fraction of wrong predictions            | Complements accuracy                                | Error rate = $1 - \text{accuracy}$ | "What is relation between accuracy and error rate?" |

## 5. Algorithm / Working Process

1. Collect true labels  $y_{true}$ .
2. Collect predicted labels  $y_{pred}$ .
3. Count how many predictions match the true labels.
4. Divide by the total number of samples.
5. Report as a decimal or percentage.

Input:

- True class labels
- Predicted class labels

Output:

- Accuracy score between 0 and 1

Training process:

- Accuracy is usually not the training loss, but it can be monitored during training.

Inference process:

- The model predicts labels.
- Accuracy compares predictions against ground truth.

## 6. Mathematical Foundation

For binary classification:

- TP = true positives
- TN = true negatives
- FP = false positives
- FN = false negatives

Formula:

$$\text{Accuracy} = (\text{TP} + \text{TN}) / (\text{TP} + \text{TN} + \text{FP} + \text{FN})$$

Error rate:

$$\text{Error rate} = 1 - \text{Accuracy} = (\text{FP} + \text{FN}) / \text{Total}$$

For multiclass classification:

$$\text{Accuracy} = \text{Number of correct predictions} / \text{Total number of predictions}$$

Accuracy estimates the probability that a random test example is classified correctly.

## 7. Practical Implementation

```
import numpy as np
from sklearn.metrics import accuracy_score

y_true = np.array([1, 0, 1, 1, 0, 2])
y_pred = np.array([1, 0, 0, 1, 0, 2])

manual_accuracy = np.mean(y_true == y_pred)
sklearn_accuracy = accuracy_score(y_true, y_pred)

print("Manual accuracy:", manual_accuracy)
print("sklearn accuracy:", sklearn_accuracy)
```

## 8. Code Explanation

- `y_true` stores actual labels.
- `y_pred` stores model predictions.
- `y_true == y_pred` returns a Boolean array.
- `np.mean(...)` treats `True` as 1 and `False` as 0.
- `accuracy_score` is the standard scikit-learn implementation.

## 9. Training / Evaluation

Accuracy is usually evaluated on validation and test sets.

Best practice:

- Use validation accuracy for model selection.
- Use test accuracy only for final reporting.
- Check class distribution before trusting accuracy.

- Compare accuracy with precision, recall, F1-score, ROC-AUC, and PR-AUC for imbalanced problems.

Accuracy may hide overfitting. A model can have high training accuracy and low validation accuracy.

## 10. Complexity and Cost

Accuracy computation is cheap.

| Cost type     | Complexity                                         |
|---------------|----------------------------------------------------|
| Time          | $O(n)$                                             |
| Memory        | $O(1)$ if streaming, $O(n)$ if storing predictions |
| CPU/GPU       | CPU is enough                                      |
| Training cost | None; metric only                                  |

## 11. Common Use Cases

- Balanced binary classification
- Multiclass image classification
- Benchmark comparison
- Quick model sanity checks
- Monitoring stable production classifiers

## 12. Common Mistakes

- Using accuracy on highly imbalanced datasets.
- Reporting only accuracy.
- Computing accuracy on training data only.
- Accidentally evaluating on leaked validation/test data.
- Comparing models on different test splits.
- Ignoring confidence calibration.

## 13. Edge Cases / Limitations

Accuracy performs poorly as a decision metric when:

- Positive class is rare.
- False positives and false negatives have different costs.
- Ranking quality matters more than hard labels.
- Model probabilities are needed.
- Top-k predictions matter.

Example: In cancer detection, missing a cancer case is much worse than a false alarm. Accuracy alone is dangerous.

## 14. Variations

| Variation          | What changes                                  | When to use                   | Placement importance |
|--------------------|-----------------------------------------------|-------------------------------|----------------------|
| Balanced accuracy  | Average recall across classes                 | Imbalanced classification     | High                 |
| Top-k accuracy     | Correct if true label is in top k predictions | Multiclass deep learning      | High                 |
| Per-class accuracy | Accuracy for each class separately            | Debugging class-wise behavior | Medium               |
| Subset accuracy    | Exact match over label sets                   | Multilabel classification     | Medium               |

## 15. Related Topics

- Confusion matrix: source table used to derive accuracy.
- Precision and recall: better when error types matter.
- F1-score: balances precision and recall.
- ROC-AUC: threshold-independent ranking metric.
- Log loss: evaluates predicted probabilities.
- Balanced accuracy: improves accuracy under imbalance.

## 16. Interview Questions

1. What is accuracy? Accuracy is the fraction of correct predictions among all predictions.
2. Give the formula for accuracy.  $(TP + TN) / (TP + TN + FP + FN)$ .
3. When is accuracy a good metric? When classes are balanced and mistake costs are similar.
4. Why is accuracy misleading for imbalanced data? A majority-class classifier can get high accuracy while ignoring minority classes.
5. What is error rate? The fraction of wrong predictions, equal to  $1 - \text{accuracy}$ .
6. Can accuracy be used for multiclass classification? Yes, it counts correct labels over total labels.
7. Why can high accuracy still be bad in medical diagnosis? Because the model may miss rare but critical positive cases.
8. What metric can replace accuracy for imbalanced data? F1-score, balanced accuracy, recall, PR-AUC, or MCC depending on the goal.
9. Is accuracy differentiable? No. It is usually not used directly as a training loss.
10. What is the difference between accuracy and log loss? Accuracy evaluates hard labels; log loss evaluates predicted probabilities.

## 17. Practice Tasks

- Small coding task: Implement accuracy from scratch using NumPy.

- Dataset project: Train logistic regression on Iris and report accuracy.
- Experiment idea: Compare accuracy on balanced vs imbalanced class distributions.
- Debugging task: Show why a majority-class classifier gets high accuracy on fraud data.
- Extension idea: Add balanced accuracy and compare results.

## 18. Project Ideas

| Project                               | What it does                                   | Tech stack                | Dataset                   | Resume value                |
|---------------------------------------|------------------------------------------------|---------------------------|---------------------------|-----------------------------|
| Balanced vs Imbalanced Classifier Lab | Shows why accuracy can fail                    | Python, scikit-learn      | Credit card fraud         | Strong metric understanding |
| Image Classifier Benchmark            | Compares models by accuracy and top-k accuracy | PyTorch, torchvision      | CIFAR-10                  | Deep learning evaluation    |
| Production Metric Dashboard           | Tracks accuracy drift over time                | Python, Streamlit, Pandas | Simulated production logs | MLOps readiness             |

## 19. Quick Revision

- Key idea: Percentage of correct predictions.
- Main formula:  $(TP + TN) / Total$ .
- When to use: Balanced classification.
- Important metrics: Accuracy, error rate, balanced accuracy.
- Common traps: Imbalanced data and unequal error costs.
- Interview one-liner: Accuracy is simple and useful, but it can hide minority-class failure.

## 20. Final Cheat Sheet

| Item                | Summary                                          |
|---------------------|--------------------------------------------------|
| Definition          | Fraction of correct predictions                  |
| Input/output        | Labels in, score between 0 and 1 out             |
| Main steps          | Compare labels, count matches, divide by total   |
| Key hyperparameters | None                                             |
| Metrics             | Accuracy, error rate                             |
| Pros                | Simple, intuitive, easy to communicate           |
| Cons                | Misleading under imbalance                       |
| Best use cases      | Balanced classification with equal mistake costs |

## Precision

## 1. Overview

Precision measures how many predicted positives are actually positive. It answers: "When the model says positive, how often is it right?"

Precision is useful when false positives are costly.

Real-world examples:

- Spam detection: Avoid marking important email as spam.
- Search retrieval: Returned documents should be relevant.
- Fraud alerts: Too many false alarms waste analyst time.
- Medical screening follow-up: Avoid unnecessary expensive tests.

## 2. Intuition

Precision is about trust in positive predictions.

Example:

If a model flags 20 transactions as fraud and 15 are truly fraud, precision is 75%.

Analogy: A security guard who stops people at an airport should stop mostly suspicious people. If they stop everyone, recall may be high, but precision is poor.

## 3. Prerequisites

- Binary classification
- Positive and negative classes
- Confusion matrix
- False positives
- Thresholding predicted probabilities

## 4. Core Concepts

| Subtopic            | Meaning                                       | Why it matters                     | Simple example                      | Interview angle                                        |
|---------------------|-----------------------------------------------|------------------------------------|-------------------------------------|--------------------------------------------------------|
| True positive       | Model predicts positive and label is positive | Good positive prediction           | Fraud correctly flagged             | "What goes in numerator?"                              |
| False positive      | Model predicts positive but label is negative | Reduces precision                  | Normal email marked spam            | "When should precision be high?"                       |
| Predicted positives | All samples predicted as positive             | Denominator                        | All flagged fraud cases             | "Precision depends on model alerts, not all positives" |
| Threshold           | Probability cutoff for positive class         | Controls precision-recall tradeoff | threshold 0.8 gives fewer positives | "How does threshold affect precision?"                 |

## 5. Algorithm / Working Process

1. Choose the positive class.
2. Generate predicted labels or probabilities.
3. If probabilities are used, choose a threshold.
4. Count true positives.
5. Count false positives.
6. Compute  $TP / (TP + FP)$ .

Input:

- True labels
- Predicted labels or probabilities
- Positive class definition

Output:

- Precision score between 0 and 1

## 6. Mathematical Foundation

$$\text{Precision} = TP / (TP + FP)$$

If there are no predicted positives:

$$TP + FP = 0$$

Precision is undefined mathematically. Libraries usually return 0 or warn depending on configuration.

Precision can also be interpreted as:

$$P(\text{actual positive} \mid \text{predicted positive})$$

## 7. Practical Implementation

```
import numpy as np
from sklearn.metrics import precision_score

y_true = np.array([1, 0, 1, 0, 1, 0, 0])
y_pred = np.array([1, 1, 1, 0, 0, 0, 1])

tp = np.sum((y_true == 1) & (y_pred == 1))
fp = np.sum((y_true == 0) & (y_pred == 1))

manual_precision = tp / (tp + fp)
```

```
sklearn_precision = precision_score(y_true, y_pred)

print("Manual precision:", manual_precision)
print("sklearn precision:", sklearn_precision)
```

## 8. Code Explanation

- `tp` counts correctly predicted positives.
- `fp` counts negatives incorrectly predicted as positives.
- Precision divides correct positive predictions by all positive predictions.
- `precision_score` handles the standard calculation.

## 9. Training / Evaluation

Precision is usually monitored on validation/test sets, especially when alerts or retrieved results are shown to users.

To improve precision:

- Increase classification threshold.
- Improve feature quality.
- Reduce label noise.
- Use class weights carefully.
- Add hard negative examples.

But increasing precision often reduces recall.

## 10. Complexity and Cost

| Cost type     | Complexity                        |
|---------------|-----------------------------------|
| Time          | $O(n)$                            |
| Memory        | $O(1)$ streaming or $O(n)$ stored |
| CPU/GPU       | CPU enough                        |
| Training cost | None                              |

## 11. Common Use Cases

- Spam filtering
- Fraud alerts
- Information retrieval
- Object detection proposal filtering
- Toxic content moderation
- Recommendation quality at top positions

## 12. Common Mistakes

- Forgetting which class is positive.
- Reporting precision without recall.
- Ignoring threshold choice.
- Comparing precision at different recall levels.
- Using macro precision without explaining averaging.
- Not handling zero predicted positives.

## 13. Edge Cases / Limitations

Precision fails to show how many actual positives were missed.

A model can achieve perfect precision by predicting only one very obvious positive sample and ignoring all others.

Precision is sensitive to:

- Threshold
- Class prevalence
- Label quality
- Definition of the positive class

## 14. Variations

| Variation          | What changes                             | When to use                       | Placement importance |
|--------------------|------------------------------------------|-----------------------------------|----------------------|
| Macro precision    | Average precision across classes equally | Multiclass imbalance              | High                 |
| Micro precision    | Compute global TP/FP                     | Large multiclass/multilabel tasks | Medium               |
| Weighted precision | Class precision weighted by support      | Imbalanced multiclass             | Medium               |
| Precision@k        | Precision among top k results            | Search/recommendation             | High                 |

## 15. Related Topics

- Recall: measures coverage of actual positives.
- F1-score: harmonic mean of precision and recall.
- PR curve: precision across thresholds.
- False discovery rate:  $1 - \text{precision}$ .
- Confusion matrix: provides TP and FP counts.

## 16. Interview Questions

1. What is precision? It is the fraction of predicted positives that are truly positive.
2. Formula?  $\text{TP} / (\text{TP} + \text{FP})$ .

3. When is precision important? When false positives are costly.
4. Give an example where precision matters more than recall. Spam filtering, where marking important email as spam is bad.
5. Can a model have high precision and low recall? Yes. It may predict positive only for very obvious cases.
6. How does increasing threshold affect precision? Usually increases precision and decreases recall.
7. What happens if there are no predicted positives? Precision is undefined; implementations often return 0 with a warning.
8. What is precision@k? Fraction of relevant items among top k retrieved results.
9. Is precision affected by true negatives? No, true negatives are not in the formula.
10. Why report precision with recall? Precision alone does not show missed positives.

## 17. Practice Tasks

- Implement precision from scratch.
- Vary threshold from 0.1 to 0.9 and plot precision.
- Evaluate precision on a spam classifier.
- Debug a classifier with high precision but poor recall.
- Compute macro and weighted precision for multiclass data.

## 18. Project Ideas

| Project                    | What it does                         | Tech stack                 | Dataset             | Resume value             |
|----------------------------|--------------------------------------|----------------------------|---------------------|--------------------------|
| Fraud Alert Optimizer      | Tunes threshold for analyst workload | scikit-learn, Pandas       | Credit card fraud   | Business-aware ML        |
| Search Precision Evaluator | Computes precision@k for retrieval   | Python, BM25/vector search | MS MARCO sample     | RAG/retrieval skill      |
| Email Spam Precision Study | Minimizes false spam alerts          | scikit-learn               | SMS Spam Collection | Practical classification |

## 19. Quick Revision

- Key idea: Correctness of positive predictions.
- Main formula:  $TP / (TP + FP)$ .
- When to use: False positives are costly.
- Important metrics: Precision, recall, F1, PR-AUC.
- Common traps: Ignoring recall and threshold.
- Interview one-liner: Precision tells how trustworthy positive predictions are.

## 20. Final Cheat Sheet

| Item                | Summary                                       |
|---------------------|-----------------------------------------------|
| Definition          | Fraction of predicted positives that are true |
| Input/output        | Labels in, precision score out                |
| Main steps          | Count TP and FP                               |
| Key hyperparameters | Decision threshold                            |
| Metrics             | Precision, precision@k                        |
| Pros                | Good for false-positive-sensitive tasks       |
| Cons                | Ignores missed positives                      |
| Best use cases      | Spam, retrieval, fraud alerts                 |

# Recall

---

## 1. Overview

Recall measures how many actual positives the model successfully finds. It answers: "Out of all true positive cases, how many did the model catch?"

Recall is useful when false negatives are costly.

Real-world examples:

- Cancer detection: Missing a cancer case is dangerous.
- Fraud detection: Missing fraud causes financial loss.
- Safety monitoring: Missing a dangerous event is unacceptable.
- Legal document discovery: Missing relevant documents can be costly.

## 2. Intuition

Recall is about coverage.

Example:

If there are 100 fraud transactions and the model catches 80, recall is 80%.

Analogy: A fishing net with high recall catches most fish, but may also catch unwanted objects. Precision tells how clean the catch is.

## 3. Prerequisites

- Positive class definition
- Confusion matrix
- True positives and false negatives
- Probability thresholding
- Class imbalance

## 4. Core Concepts

| Subtopic         | Meaning                            | Why it matters               | Simple example        | Interview angle                                             |
|------------------|------------------------------------|------------------------------|-----------------------|-------------------------------------------------------------|
| True positive    | Actual positive correctly detected | Increases recall             | Sick patient detected | "What is numerator?"                                        |
| False negative   | Actual positive missed by model    | Reduces recall               | Fraud marked safe     | "When is FN costly?"                                        |
| Actual positives | All positive cases in data         | Denominator                  | All real frauds       | "Recall depends on true positives, not predicted positives" |
| Sensitivity      | Another name for recall            | Common in medical statistics | Test sensitivity      | "Recall vs sensitivity?"                                    |

## 5. Algorithm / Working Process

1. Identify positive class.
2. Predict labels using a chosen threshold.
3. Count true positives.
4. Count false negatives.
5. Compute  $TP / (TP + FN)$ .

Input:

- True labels
- Predicted labels or scores
- Positive class

Output:

- Recall score between 0 and 1

## 6. Mathematical Foundation

$$\text{Recall} = TP / (TP + FN)$$

Recall is also called:

$$\text{Sensitivity} = \text{True Positive Rate} = \text{TPR}$$

Probability interpretation:

$P(\text{predicted positive} \mid \text{actual positive})$

False negative rate:

$$\text{FNR} = \text{FN} / (\text{TP} + \text{FN}) = 1 - \text{Recall}$$

## 7. Practical Implementation

```
import numpy as np
from sklearn.metrics import recall_score

y_true = np.array([1, 0, 1, 1, 0, 1, 0])
y_pred = np.array([1, 0, 0, 1, 0, 1, 1])

tp = np.sum((y_true == 1) & (y_pred == 1))
fn = np.sum((y_true == 1) & (y_pred == 0))

manual_recall = tp / (tp + fn)
sklearn_recall = recall_score(y_true, y_pred)

print("Manual recall:", manual_recall)
print("sklearn recall:", sklearn_recall)
```

## 8. Code Explanation

- `tp` counts positive samples correctly detected.
- `fn` counts positive samples missed.
- Recall divides detected positives by all actual positives.
- `recall_score` implements the metric directly.

## 9. Training / Evaluation

Recall is important for validation when missing positives is expensive.

To improve recall:

- Lower the classification threshold.
- Use class weighting or oversampling.
- Collect more positive examples.
- Improve labels for rare positive cases.
- Use anomaly detection or cascaded models for rare events.

Tradeoff:

- Higher recall often lowers precision.

## 10. Complexity and Cost

| Cost type     | Complexity                        |
|---------------|-----------------------------------|
| Time          | $O(n)$                            |
| Memory        | $O(1)$ streaming or $O(n)$ stored |
| CPU/GPU       | CPU enough                        |
| Training cost | None                              |

## 11. Common Use Cases

- Medical diagnosis
- Fraud detection
- Intrusion detection
- Safety-critical vision systems
- Defect detection in manufacturing
- High-risk content moderation

## 12. Common Mistakes

- Optimizing recall without checking precision.
- Forgetting that lowering threshold increases alerts.
- Reporting recall on a biased test set.
- Ignoring false positive cost.
- Confusing recall with precision.
- Using accuracy instead of recall for rare positive classes.

## 13. Edge Cases / Limitations

Recall does not penalize false positives.

A model can achieve 100% recall by predicting every sample as positive. This is useless if precision becomes very low.

Recall depends heavily on:

- Threshold
- Positive class definition
- Test set representativeness
- Label completeness

## 14. Variations

| Variation    | What changes             | When to use                    | Placement importance |
|--------------|--------------------------|--------------------------------|----------------------|
| Macro recall | Average recall per class | Imbalanced multiclass          | High                 |
| Micro recall | Global TP/FN calculation | Multilabel or large multiclass | Medium               |

| Variation       | What changes               | When to use                 | Placement importance |
|-----------------|----------------------------|-----------------------------|----------------------|
| Weighted recall | Weighted by class support  | Imbalanced multiclass       | Medium               |
| Recall@k        | True item appears in top k | Search, recommendation, RAG | High                 |
| Specificity     | Recall of negative class   | Medical testing             | High                 |

## 15. Related Topics

- Precision: measures correctness of positive predictions.
- F1-score: combines precision and recall.
- ROC curve: plots recall/TPR against FPR.
- PR curve: plots precision against recall.
- Confusion matrix: source of TP and FN.

## 16. Interview Questions

1. What is recall? Fraction of actual positives correctly detected.
2. Formula?  $TP / (TP + FN)$ .
3. When is recall important? When false negatives are costly.
4. Give an example where recall matters more than precision. Cancer screening.
5. What is another name for recall? Sensitivity or true positive rate.
6. How does lowering threshold affect recall? Usually increases recall.
7. Can recall be 100% and model still bad? Yes, if it predicts everything positive.
8. Does recall use true negatives? No.
9. What is false negative rate?  $1 - \text{recall}$ .
10. Why use PR curve with recall? It shows precision-recall tradeoff across thresholds.

## 17. Practice Tasks

- Implement recall from scratch.
- Tune threshold for 95% recall.
- Train a fraud classifier and compare recall vs precision.
- Plot recall across thresholds.
- Compute macro recall on multiclass data.

## 18. Project Ideas

| Project                      | What it does                            | Tech stack              | Dataset                 | Resume value                   |
|------------------------------|-----------------------------------------|-------------------------|-------------------------|--------------------------------|
| Medical Screening Classifier | Maximizes recall for disease detection  | scikit-learn            | Breast Cancer Wisconsin | Healthcare ML metric reasoning |
| Fraud Recall Optimizer       | Tunes threshold to catch fraud          | Python, XGBoost/sklearn | Credit card fraud       | Imbalanced ML project          |
| RAG Recall@k Evaluator       | Checks if relevant context is retrieved | Python, FAISS           | QA dataset              | Strong AI engineering value    |

## 19. Quick Revision

- Key idea: Coverage of actual positives.
- Main formula:  $TP / (TP + FN)$ .
- When to use: False negatives are costly.
- Important metrics: Recall, precision, F1, PR-AUC.
- Common traps: Predicting everything positive.
- Interview one-liner: Recall tells how many real positives the model catches.

## 20. Final Cheat Sheet

| Item                | Summary                               |
|---------------------|---------------------------------------|
| Definition          | Fraction of actual positives detected |
| Input/output        | Labels in, recall score out           |
| Main steps          | Count TP and FN                       |
| Key hyperparameters | Decision threshold                    |
| Metrics             | Recall, FNR, sensitivity              |
| Pros                | Captures missed-positive risk         |
| Cons                | Ignores false positives               |
| Best use cases      | Medical, fraud, safety systems        |

## F1-score

### 1. Overview

F1-score is the harmonic mean of precision and recall. It gives one number that balances false positives and false negatives.

It is useful when:

- Classes are imbalanced.
- Both false positives and false negatives matter.

- You need a single metric but accuracy is misleading.

Real-world usage:

- Spam detection.
- Fraud detection.
- NLP classification.
- Named entity recognition.
- Information extraction.

## 2. Intuition

F1 rewards models that have both good precision and good recall.

If precision is high but recall is low, F1 is low. If recall is high but precision is low, F1 is low.

Analogy: A good search engine should return relevant results and find most relevant results. F1 balances both.

## 3. Prerequisites

- Precision
- Recall
- Harmonic mean
- Confusion matrix
- Class imbalance
- Threshold tuning

## 4. Core Concepts

| Subtopic      | Meaning                             | Why it matters                       | Simple example               | Interview angle           |
|---------------|-------------------------------------|--------------------------------------|------------------------------|---------------------------|
| Precision     | Correctness of positive predictions | Penalizes false positives            | Flagged fraud is truly fraud | "What does F1 combine?"   |
| Recall        | Coverage of actual positives        | Penalizes false negatives            | Real fraud is caught         | "Why not only precision?" |
| Harmonic mean | Mean that punishes imbalance        | Prevents one metric hiding the other | $P=1, R=0$ gives $F1=0$      | "Why harmonic mean?"      |
| F-beta        | Weighted F-score                    | Emphasizes precision or recall       | F2 favors recall             | "When use F2?"            |

## 5. Algorithm / Working Process

1. Compute precision.
2. Compute recall.
3. Combine them using harmonic mean.
4. For multiclass, choose averaging method: macro, micro, or weighted.

Input:

- True labels
- Predicted labels

Output:

- F1-score between 0 and 1

## 6. Mathematical Foundation

```
Precision = TP / (TP + FP)
Recall = TP / (TP + FN)
F1 = 2 * Precision * Recall / (Precision + Recall)
```

Equivalent binary formula:

```
F1 = 2TP / (2TP + FP + FN)
```

F-beta score:

```
F_beta = (1 + beta^2) * (Precision * Recall) / (beta^2 * Precision + Recall)
```

- $\beta > 1$ : recall matters more.
- $\beta < 1$ : precision matters more.

## 7. Practical Implementation

```
import numpy as np
from sklearn.metrics import f1_score, precision_score, recall_score

y_true = np.array([1, 0, 1, 1, 0, 0, 1])
y_pred = np.array([1, 0, 0, 1, 0, 1, 1])

precision = precision_score(y_true, y_pred)
recall = recall_score(y_true, y_pred)
manual_f1 = 2 * precision * recall / (precision + recall)

print("Precision:", precision)
print("Recall:", recall)
print("Manual F1:", manual_f1)
print("sklearn F1:", f1_score(y_true, y_pred))
```

## 8. Code Explanation

- `precision_score` calculates positive prediction correctness.

- `recall_score` calculates positive class coverage.
- F1 is manually computed from precision and recall.
- `f1_score` confirms the library result.

## 9. Training / Evaluation

F1 is commonly used for validation-based threshold selection.

Workflow:

1. Train model using a differentiable loss such as log loss.
2. Predict probabilities on validation data.
3. Try multiple thresholds.
4. Select threshold with best F1.
5. Report final F1 on test data.

F1 is not usually optimized directly during training because hard predictions and counts are not differentiable.

## 10. Complexity and Cost

| Cost type     | Complexity                        |
|---------------|-----------------------------------|
| Time          | $O(n)$                            |
| Memory        | $O(1)$ streaming or $O(n)$ stored |
| CPU/GPU       | CPU enough                        |
| Training cost | None                              |

## 11. Common Use Cases

- Imbalanced classification
- Spam detection
- Fraud detection
- Text classification
- Named entity recognition
- Object detection class-level evaluation

## 12. Common Mistakes

- Using F1 when true negatives matter a lot.
- Ignoring probability calibration.
- Comparing binary F1 with macro F1 without saying which one.
- Using default threshold blindly.
- Reporting F1 without class distribution.
- Assuming F1 is always better than accuracy.

## 13. Edge Cases / Limitations

F1 ignores true negatives. This can be a problem when correct negative classification is important.

F1 also depends on threshold. A model with better probability ranking can have worse F1 if the threshold is poorly chosen.

F1 is less interpretable to business users than precision and recall separately.

## 14. Variations

| Variation   | What changes                 | When to use                     | Placement importance |
|-------------|------------------------------|---------------------------------|----------------------|
| Macro F1    | Equal average across classes | Imbalanced multiclass           | Very high            |
| Micro F1    | Global TP/FP/FN              | Multilabel, large class sets    | High                 |
| Weighted F1 | Weighted by class frequency  | Imbalanced multiclass reporting | High                 |
| F0.5        | Precision weighted more      | False positives cost more       | Medium               |
| F2          | Recall weighted more         | False negatives cost more       | Medium               |

## 15. Related Topics

- Precision: one half of F1.
- Recall: one half of F1.
- PR curve: shows precision-recall tradeoff.
- Threshold tuning: often needed to maximize F1.
- MCC: alternative metric using all confusion matrix entries.

## 16. Interview Questions

1. What is F1-score? The harmonic mean of precision and recall.
2. Formula?  $2PR / (P + R)$ .
3. Why harmonic mean? It penalizes imbalance between precision and recall.
4. When is F1 useful? For imbalanced data where FP and FN both matter.
5. Can F1 be high if precision is low? No, low precision pulls F1 down.
6. Does F1 use true negatives? No.
7. What is macro F1? Average of per-class F1 scores, treating classes equally.
8. What is weighted F1? Average of per-class F1 weighted by support.
9. Difference between F1 and accuracy? Accuracy counts all correct predictions; F1 balances precision and recall for positives.
10. When use F2? When recall is more important than precision.

## 17. Practice Tasks

- Implement F1 from scratch.
- Tune threshold to maximize F1.

- Compare F1 and accuracy on imbalanced data.
- Compute macro F1 on a multiclass dataset.
- Analyze a confusion matrix where F1 is misleading.

## 18. Project Ideas

| Project                             | What it does                                   | Tech stack                   | Dataset            | Resume value                  |
|-------------------------------------|------------------------------------------------|------------------------------|--------------------|-------------------------------|
| Imbalanced Classification Benchmark | Compares accuracy, F1, MCC                     | scikit-learn                 | Fraud dataset      | Metric selection skill        |
| NLP Intent Classifier               | Reports macro F1 by intent                     | Python, transformers/sklearn | Banking77          | NLP evaluation                |
| Threshold Tuning App                | Interactive precision-recall-F1 threshold tool | Streamlit, scikit-learn      | Any binary dataset | Practical deployment thinking |

## 19. Quick Revision

- Key idea: Balance precision and recall.
- Main formula:  $2PR / (P + R)$ .
- When to use: Imbalanced classification.
- Important metrics: Precision, recall, macro F1.
- Common traps: Ignoring true negatives and averaging type.
- Interview one-liner: F1 is useful when both false positives and false negatives matter.

## 20. Final Cheat Sheet

| Item                | Summary                                     |
|---------------------|---------------------------------------------|
| Definition          | Harmonic mean of precision and recall       |
| Input/output        | Labels in, F1 score out                     |
| Main steps          | Compute precision, recall, combine          |
| Key hyperparameters | Threshold, averaging method                 |
| Metrics             | F1, macro F1, weighted F1, F-beta           |
| Pros                | Better than accuracy for imbalance          |
| Cons                | Ignores true negatives                      |
| Best use cases      | NLP, fraud, spam, rare-event classification |

## Confusion Matrix

### 1. Overview

A confusion matrix is a table that compares actual labels with predicted labels. It is the foundation for many classification metrics, including accuracy, precision, recall, F1-score, specificity, and MCC.

It is useful because it shows exactly what kinds of mistakes a model makes.

Real-world usage:

- Debugging classification models.
- Understanding class-wise errors.
- Explaining model performance to stakeholders.
- Comparing false positive and false negative costs.

2. Intuition

The confusion matrix is a scoreboard of predictions.

For binary classification:

|                 | Predicted Positive | Predicted Negative |
|-----------------|--------------------|--------------------|
| Actual Positive | True Positive      | False Negative     |
| Actual Negative | False Positive     | True Negative      |

It tells whether the model is confusing one class with another.

3. Prerequisites

- Classification labels
- Predicted labels
- Binary and multiclass classification
- Basic matrix/table reading
- Error analysis

4. Core Concepts

| Subtopic          | Meaning                             | Why it matters             | Simple example              | Interview angle               |
|-------------------|-------------------------------------|----------------------------|-----------------------------|-------------------------------|
| TP                | Actual positive, predicted positive | Correct positive detection | Fraud caught                | "Define TP"                   |
| TN                | Actual negative, predicted negative | Correct negative detection | Normal transaction accepted | "Define TN"                   |
| FP                | Actual negative, predicted positive | False alarm                | Normal marked fraud         | "Which metric penalizes FP?"  |
| FN                | Actual positive, predicted negative | Missed positive            | Fraud missed                | "Which metric penalizes FN?"  |
| Multiclass matrix | Rows/classes vs predicted classes   | Shows class confusion      | Cat predicted as dog        | "How read multiclass matrix?" |

## 5. Algorithm / Working Process

1. List all possible classes.
2. Create a square matrix of size `num_classes x num_classes`.
3. For each example, find actual class row and predicted class column.
4. Increment that cell.
5. Analyze diagonal and off-diagonal values.

Input:

- True labels
- Predicted labels

Output:

- Count matrix

## 6. Mathematical Foundation

For binary classification:

```
TP = count(y_true = 1 and y_pred = 1)
TN = count(y_true = 0 and y_pred = 0)
FP = count(y_true = 0 and y_pred = 1)
FN = count(y_true = 1 and y_pred = 0)
```

For multiclass classification:

```
C[i, j] = number of samples whose true class is i and predicted class is j
```

The diagonal values `C[i, i]` are correct predictions.

## 7. Practical Implementation

```
import numpy as np
from sklearn.metrics import confusion_matrix, ConfusionMatrixDisplay
import matplotlib.pyplot as plt

y_true = np.array(["cat", "dog", "cat", "bird", "dog", "bird"])
y_pred = np.array(["cat", "cat", "cat", "bird", "dog", "dog"])

labels = ["bird", "cat", "dog"]
cm = confusion_matrix(y_true, y_pred, labels=labels)

print(cm)

disp = ConfusionMatrixDisplay(confusion_matrix=cm, display_labels=labels)
```

```
disp.plot(cmap="Blues")
plt.title("Confusion Matrix")
plt.show()
```

## 8. Code Explanation

- `labels` fixes row/column ordering.
- `confusion_matrix` counts true-vs-predicted pairs.
- Rows represent actual labels.
- Columns represent predicted labels in scikit-learn's default convention.
- `ConfusionMatrixDisplay` visualizes the matrix.

## 9. Training / Evaluation

Use confusion matrices during validation and final testing.

Good workflow:

- Train model.
- Predict validation labels.
- Plot confusion matrix.
- Identify repeated class confusions.
- Improve data, features, labels, or threshold based on errors.

For imbalanced data, normalize rows to inspect per-class recall.

## 10. Complexity and Cost

| Cost type     | Complexity               |
|---------------|--------------------------|
| Time          | $O(n)$                   |
| Memory        | $O(k^2)$ for $k$ classes |
| CPU/GPU       | CPU enough               |
| Training cost | None                     |

Large label spaces like extreme classification can produce huge matrices.

## 11. Common Use Cases

- Binary classification diagnosis
- Multiclass error analysis
- Medical model validation
- Computer vision class confusion
- NLP intent classification
- Model monitoring dashboards

## 12. Common Mistakes

- Mixing up rows and columns.
- Not specifying label order.
- Reading raw counts without considering class imbalance.
- Reporting confusion matrix on training data only.
- Ignoring off-diagonal patterns.
- Comparing matrices from different test distributions.

## 13. Edge Cases / Limitations

Confusion matrices can become hard to read when the number of classes is large.

They also depend on a fixed threshold. They do not directly show probability quality or ranking quality.

For multilabel classification, each label may need its own binary confusion matrix.

## 14. Variations

| Variation                   | What changes            | When to use               | Placement importance |
|-----------------------------|-------------------------|---------------------------|----------------------|
| Normalized by true labels   | Rows sum to 1           | Per-class recall analysis | High                 |
| Normalized by predictions   | Columns sum to 1        | Precision-style analysis  | Medium               |
| Binary one-vs-rest matrix   | One class vs all others | Multiclass class analysis | High                 |
| Multilabel confusion matrix | One matrix per label    | Multilabel tasks          | Medium               |

## 15. Related Topics

- Accuracy: diagonal sum divided by total.
- Precision: uses TP and FP.
- Recall: uses TP and FN.
- F1-score: combines precision and recall.
- MCC: uses all four binary entries.
- Cohen's kappa: compares accuracy against chance agreement.

## 16. Interview Questions

1. What is a confusion matrix? A table comparing actual and predicted labels.
2. What are TP, TN, FP, FN? The four outcomes in binary classification.
3. Which axis is true label in scikit-learn? Rows are true labels, columns are predicted labels.
4. How do you compute accuracy from it? Sum diagonal values and divide by total values.
5. How do you compute precision?  $TP / (TP + FP)$ .
6. How do you compute recall?  $TP / (TP + FN)$ .
7. What does an off-diagonal entry mean? A class was confused with another class.
8. Why normalize a confusion matrix? To compare classes with different sample counts.

9. How does multiclass confusion matrix differ? It has one row and column per class.

10. What is a limitation? It does not evaluate probability calibration or ranking.

## 17. Practice Tasks

- Build a confusion matrix manually.
- Plot a normalized confusion matrix for Iris.
- Identify the most confused class pair.
- Compute accuracy, precision, recall, and F1 from matrix entries.
- Create one-vs-rest confusion matrices for multiclass labels.

## 18. Project Ideas

| Project                  | What it does                  | Tech stack         | Dataset                 | Resume value                   |
|--------------------------|-------------------------------|--------------------|-------------------------|--------------------------------|
| Error Analysis Dashboard | Visualizes confusion by class | Streamlit, sklearn | CIFAR-10 or Iris        | Strong evaluation storytelling |
| Medical Test Analyzer    | Shows TP/TN/FP/FN tradeoffs   | Python, Pandas     | Breast Cancer Wisconsin | Applied metric reasoning       |
| NLP Intent Debugger      | Finds confused intent pairs   | Python, sklearn    | Banking77               | NLP production evaluation      |

## 19. Quick Revision

- Key idea: Table of actual vs predicted labels.
- Main formula: `C[i, j] = count(true=i, pred=j)`.
- When to use: Error analysis.
- Important metrics: Accuracy, precision, recall, F1.
- Common traps: Axis confusion and class imbalance.
- Interview one-liner: A confusion matrix shows not just how often the model is wrong, but how it is wrong.

## 20. Final Cheat Sheet

| Item                | Summary                                     |
|---------------------|---------------------------------------------|
| Definition          | Count table of true vs predicted labels     |
| Input/output        | Labels in, matrix out                       |
| Main steps          | Count true-predicted class pairs            |
| Key hyperparameters | Label order, threshold                      |
| Metrics             | Accuracy, precision, recall, F1, MCC        |
| Pros                | Great for error analysis                    |
| Cons                | Hard with many classes; threshold-dependent |

| Item           | Summary                                |
|----------------|----------------------------------------|
| Best use cases | Classification debugging and reporting |

# ROC Curve

---

## 1. Overview

The ROC curve, or Receiver Operating Characteristic curve, shows how a binary classifier behaves across different decision thresholds. It plots true positive rate against false positive rate.

It is useful when:

- You care about ranking positive samples above negative samples.
- You want threshold-independent model comparison.
- Class distribution may change, but ranking quality matters.

Real-world usage:

- Medical diagnosis threshold selection.
- Fraud risk scoring.
- Credit scoring.
- Binary classifier comparison.

## 2. Intuition

A classifier often outputs probabilities or scores, not just labels. Changing the threshold changes how many positives it predicts.

Low threshold:

- More positives caught.
- More false alarms.

High threshold:

- Fewer false alarms.
- More missed positives.

ROC curve shows this tradeoff.

## 3. Prerequisites

- Binary classification
- Probability scores
- Thresholding
- Confusion matrix
- Recall/TPR
- False positive rate

## 4. Core Concepts

| Subtopic  | Meaning                        | Why it matters               | Simple example                  | Interview angle                        |
|-----------|--------------------------------|------------------------------|---------------------------------|----------------------------------------|
| TPR       | $TP / (TP + FN)$               | Y-axis of ROC                | Caught fraud fraction           | "TPR is recall"                        |
| FPR       | $FP / (FP + TN)$               | X-axis of ROC                | False alarms among normal cases | "How is FPR different from precision?" |
| Threshold | Cutoff for positive prediction | Generates curve points       | threshold 0.7                   | "Why ROC has many points?"             |
| Ranking   | Ordering by model score        | ROC measures ranking quality | Fraud should score higher       | "ROC-AUC interpretation?"              |

## 5. Algorithm / Working Process

1. Get true binary labels.
2. Get predicted scores or probabilities.
3. Sort possible thresholds.
4. For each threshold:
  - Convert scores to labels.
  - Compute TPR.
  - Compute FPR.
5. Plot FPR on x-axis and TPR on y-axis.

Input:

- True binary labels
- Prediction scores for positive class

Output:

- ROC points: FPR, TPR, thresholds

## 6. Mathematical Foundation

$$\begin{aligned} \text{TPR} &= TP / (TP + FN) \\ \text{FPR} &= FP / (FP + TN) \end{aligned}$$

ROC curve:

$$\text{ROC} = \text{set of points } (FPR(t), TPR(t)) \text{ for thresholds } t$$

Important reference lines:

- Perfect classifier: curve goes near top-left.

- Random classifier: diagonal line from  $(0, 0)$  to  $(1, 1)$ .

## 7. Practical Implementation

```
import numpy as np
from sklearn.metrics import roc_curve, auc
import matplotlib.pyplot as plt

y_true = np.array([0, 0, 1, 1, 0, 1])
y_score = np.array([0.05, 0.30, 0.40, 0.90, 0.20, 0.80])

fpr, tpr, thresholds = roc_curve(y_true, y_score)
roc_auc = auc(fpr, tpr)

plt.plot(fpr, tpr, label=f"ROC AUC = {roc_auc:.2f}")
plt.plot([0, 1], [0, 1], linestyle="--", label="Random")
plt.xlabel("False Positive Rate")
plt.ylabel("True Positive Rate")
plt.title("ROC Curve")
plt.legend()
plt.show()
```

## 8. Code Explanation

- `y_score` contains model confidence for the positive class.
- `roc_curve` evaluates many thresholds.
- `fpr` and `tpr` form the curve.
- `auc` calculates area under the curve.
- The dashed diagonal represents random guessing.

## 9. Training / Evaluation

ROC is evaluated after training using validation/test scores.

Use ROC curve to:

- Compare model ranking ability.
- Select operating threshold.
- Understand TPR/FPR tradeoff.
- Report AUC with the curve.

For highly imbalanced datasets, PR curve is often more informative.

## 10. Complexity and Cost

| Cost type | Complexity                         |
|-----------|------------------------------------|
| Time      | $O(n \log n)$ due to score sorting |
| Memory    | $O(n)$                             |

| Cost type     | Complexity |
|---------------|------------|
| CPU/GPU       | CPU enough |
| Training cost | None       |

## 11. Common Use Cases

- Medical screening
- Fraud detection
- Risk scoring
- Binary classifier comparison
- Threshold selection
- Credit default prediction

## 12. Common Mistakes

- Using predicted hard labels instead of scores.
- Trusting ROC-AUC on extreme imbalance without PR curve.
- Interpreting FPR as false discovery rate.
- Ignoring the actual deployment threshold.
- Comparing ROC curves on different test sets.
- Thinking ROC-AUC measures calibration.

## 13. Edge Cases / Limitations

ROC can look strong on imbalanced data even when precision is poor.

Example:

If negatives are huge in number, a small FPR can still produce many false positives.

ROC also does not tell whether predicted probabilities are calibrated.

## 14. Variations

| Variation      | What changes                             | When to use                  | Placement importance |
|----------------|------------------------------------------|------------------------------|----------------------|
| Multiclass ROC | One-vs-rest or one-vs-one                | Multiclass classifiers       | Medium               |
| Partial AUC    | Area over limited FPR range              | Low false alarm applications | Medium               |
| DET curve      | Uses error rates on normal deviate scale | Biometrics/speech            | Low-medium           |
| Cost curve     | Includes cost assumptions                | Business decision making     | Medium               |

## 15. Related Topics

- AUC: scalar summary of ROC.
- Recall: same as TPR.
- Specificity:  $TN / (TN + FP)$ , so  $FPR = 1 - \text{specificity}$ .
- PR curve: better under severe imbalance.
- Calibration: separate from ranking.

## 16. Interview Questions

1. What does ROC curve plot? TPR against FPR across thresholds.
2. What is TPR? Recall,  $TP / (TP + FN)$ .
3. What is FPR?  $FP / (FP + TN)$ .
4. Why use scores instead of labels? ROC needs threshold variation.
5. What does the diagonal mean? Random classifier behavior.
6. What is a perfect ROC curve? It reaches top-left with AUC 1.
7. Is ROC good for imbalanced data? It can be less informative; PR curve may be better.
8. Does ROC measure calibration? No, it measures ranking behavior.
9. How choose threshold from ROC? Based on acceptable TPR/FPR or business cost.
10. Can two models with same AUC behave differently? Yes, their curves may differ in the operating region.

## 17. Practice Tasks

- Plot ROC for logistic regression.
- Compare ROC curves for two classifiers.
- Pick threshold for FPR below 5%.
- Show ROC vs PR curve on imbalanced data.
- Implement TPR/FPR calculation manually.

## 18. Project Ideas

| Project                    | What it does                           | Tech stack              | Dataset       | Resume value               |
|----------------------------|----------------------------------------|-------------------------|---------------|----------------------------|
| ROC Threshold Lab          | Lets users choose operating thresholds | Streamlit, sklearn      | Breast cancer | Excellent interview demo   |
| Credit Risk ROC Study      | Compares scoring models                | Python, XGBoost/sklearn | German Credit | Applied finance ML         |
| Imbalance ROC vs PR Report | Demonstrates metric differences        | Pandas, matplotlib      | Fraud data    | Strong evaluation maturity |

## 19. Quick Revision

- Key idea: TPR vs FPR across thresholds.

- Main formula:  $TPR = TP / (TP + FN)$ ,  $FPR = FP / (FP + TN)$ .
- When to use: Ranking binary classifiers.
- Important metrics: ROC-AUC, TPR, FPR.
- Common traps: Using ROC alone on imbalance.
- Interview one-liner: ROC shows how recall improves as false alarms increase.

## 20. Final Cheat Sheet

| Item                | Summary                               |
|---------------------|---------------------------------------|
| Definition          | Curve of TPR vs FPR                   |
| Input/output        | True labels and scores in, curve out  |
| Main steps          | Sweep threshold, compute TPR/FPR      |
| Key hyperparameters | Threshold range, positive class       |
| Metrics             | TPR, FPR, AUC                         |
| Pros                | Threshold-independent ranking view    |
| Cons                | Can mislead on rare positives         |
| Best use cases      | Binary ranking and threshold analysis |

## AUC

### 1. Overview

AUC means Area Under the Curve. In classification interviews, it usually refers to ROC-AUC: the area under the ROC curve.

ROC-AUC summarizes ranking performance in one number. It measures how well the model ranks positive examples above negative examples.

It is useful when:

- You need threshold-independent comparison.
- Model outputs probabilities or scores.
- Ranking quality matters.

### 2. Intuition

ROC-AUC can be interpreted as:

If you randomly pick one positive example and one negative example, AUC is the probability that the model gives the positive example a higher score.

Example:

An AUC of 0.90 means there is a 90% chance a random positive is scored higher than a random negative.

### 3. Prerequisites

- ROC curve
- TPR and FPR
- Probability scores
- Ranking
- Thresholds
- Binary classification

### 4. Core Concepts

| Subtopic            | Meaning                        | Why it matters                | Simple example             | Interview angle            |
|---------------------|--------------------------------|-------------------------------|----------------------------|----------------------------|
| Area                | Integral under ROC curve       | Converts curve to scalar      | AUC = 0.85                 | "What does AUC summarize?" |
| Ranking probability | Positive ranked above negative | Gives intuitive meaning       | Fraud score > normal score | "AUC interpretation?"      |
| Random baseline     | AUC = 0.5                      | No ranking skill              | Coin flip                  | "What is bad AUC?"         |
| Perfect ranking     | AUC = 1.0                      | All positives above negatives | Ideal classifier           | "What is perfect AUC?"     |

### 5. Algorithm / Working Process

1. Get true labels.
2. Get predicted positive scores.
3. Compute ROC curve by sweeping thresholds.
4. Approximate area under the curve, usually with trapezoidal rule.
5. Return AUC score.

Input:

- True binary labels
- Predicted scores

Output:

- AUC score, usually between 0 and 1

### 6. Mathematical Foundation

ROC-AUC:

$$\text{AUC} = \text{integral from } 0 \text{ to } 1 \text{ of } \text{TPR}(\text{FPR}) \, d(\text{FPR})$$

Trapezoidal approximation:

$$\text{AUC} \approx \sum_i (x_i - x_{i-1}) * (y_i + y_{i-1}) / 2$$

where:

- $x$  = FPR
- $y$  = TPR

Pairwise ranking interpretation:

$$\text{AUC} = P(\text{score}_{\text{positive}} > \text{score}_{\text{negative}})$$

with tie handling:

$$\text{AUC} = P(\text{pos} > \text{neg}) + 0.5 * P(\text{pos} = \text{neg})$$

## 7. Practical Implementation

```
import numpy as np
from sklearn.metrics import roc_auc_score

y_true = np.array([0, 0, 1, 1, 0, 1])
y_score = np.array([0.10, 0.35, 0.40, 0.95, 0.20, 0.80])

auc_score = roc_auc_score(y_true, y_score)

print("ROC-AUC:", auc_score)
```

## 8. Code Explanation

- `y_true` contains binary labels.
- `y_score` contains positive-class scores.
- `roc_auc_score` sorts examples by score and computes ranking quality.
- Hard labels should not be used unless no scores are available.

## 9. Training / Evaluation

AUC is used during validation for model comparison.

Useful workflow:

- Train several classifiers.
- Evaluate ROC-AUC on validation data.
- Pick model with strong AUC in the required operating region.

- Tune threshold separately using precision/recall/cost.

AUC is not a deployment threshold. It only summarizes ranking.

## 10. Complexity and Cost

| Cost type     | Complexity                   |
|---------------|------------------------------|
| Time          | $O(n \log n)$ due to sorting |
| Memory        | $O(n)$                       |
| CPU/GPU       | CPU enough                   |
| Training cost | None                         |

## 11. Common Use Cases

- Binary classifier comparison
- Medical risk scoring
- Credit default prediction
- Fraud ranking
- Model leaderboard metrics
- Early-stage model selection

## 12. Common Mistakes

- Using AUC as proof that thresholded predictions are good.
- Ignoring PR-AUC on rare positive problems.
- Using predicted labels instead of probabilities.
- Assuming AUC measures calibration.
- Not checking confidence intervals.
- Reporting AUC without business threshold performance.

## 13. Edge Cases / Limitations

AUC can hide poor performance in the region you actually care about.

Example:

A fraud team may require FPR below 1%. A model can have high overall AUC but poor TPR at FPR below 1%.

For highly imbalanced data, ROC-AUC can look good while precision is low.

## 14. Variations

| Variation | What changes        | When to use            | Placement importance |
|-----------|---------------------|------------------------|----------------------|
| ROC-AUC   | Area under ROC      | General binary ranking | Very high            |
| PR-AUC    | Area under PR curve | Rare positive class    | Very high            |

| Variation   | What changes                | When to use                       | Placement importance |
|-------------|-----------------------------|-----------------------------------|----------------------|
| Partial AUC | Area over limited FPR range | Strict false-positive constraints | Medium               |
| Macro AUC   | Average one-vs-rest AUC     | Multiclass classification         | Medium               |
| Micro AUC   | Global pooled AUC           | Multilabel tasks                  | Medium               |

## 15. Related Topics

- ROC curve: AUC summarizes it.
- PR curve: better when positives are rare.
- Log loss: evaluates probability quality, not just ranking.
- Calibration: AUC can be high even when probabilities are poorly calibrated.
- Mann-Whitney U statistic: mathematically related to AUC.

## 16. Interview Questions

1. What is AUC? Area under a performance curve, usually ROC curve.
2. What does ROC-AUC measure? Ranking quality between positive and negative examples.
3. Interpret AUC = 0.8. A random positive is ranked above a random negative about 80% of the time.
4. What is random AUC? 0.5.
5. What is perfect AUC? 1.0.
6. Can AUC be below 0.5? Yes, the model ranks worse than random; flipping scores may improve it.
7. Does AUC depend on threshold? No, it summarizes all thresholds.
8. Does AUC measure calibration? No.
9. When prefer PR-AUC over ROC-AUC? For highly imbalanced datasets with rare positives.
10. What is partial AUC? AUC computed over a restricted FPR or TPR region.

## 17. Practice Tasks

- Compute ROC-AUC using scikit-learn.
- Manually calculate AUC from ROC points.
- Compare AUC and F1 under different thresholds.
- Plot partial ROC for low-FPR range.
- Compare ROC-AUC and PR-AUC on imbalanced data.

## 18. Project Ideas

| Project                        | What it does                    | Tech stack       | Dataset       | Resume value              |
|--------------------------------|---------------------------------|------------------|---------------|---------------------------|
| Binary Model Ranking Benchmark | Compares AUC across classifiers | sklearn, XGBoost | Breast cancer | Classic placement project |

| Project                   | What it does                          | Tech stack          | Dataset           | Resume value              |
|---------------------------|---------------------------------------|---------------------|-------------------|---------------------------|
| Fraud AUC vs PR-AUC Study | Shows metric behavior under imbalance | Python, Pandas      | Credit card fraud | Strong metric judgment    |
| Partial AUC Medical Tool  | Evaluates low-FPR screening region    | sklearn, matplotlib | Medical dataset   | Research-style evaluation |

## 19. Quick Revision

- Key idea: Area under ROC curve.
- Main formula:  $\int \text{TPR} \, d\text{FPR}$ .
- When to use: Ranking model comparison.
- Important metrics: ROC-AUC, PR-AUC, partial AUC.
- Common traps: Confusing AUC with calibrated probability quality.
- Interview one-liner: ROC-AUC is the probability a random positive gets a higher score than a random negative.

## 20. Final Cheat Sheet

| Item                | Summary                                  |
|---------------------|------------------------------------------|
| Definition          | Area under ROC curve                     |
| Input/output        | Labels and scores in, scalar score out   |
| Main steps          | Sort scores, build ROC, integrate        |
| Key hyperparameters | Positive class, averaging type           |
| Metrics             | ROC-AUC, PR-AUC                          |
| Pros                | Threshold-independent ranking metric     |
| Cons                | Can hide poor precision under imbalance  |
| Best use cases      | Binary risk scoring and model comparison |

# PR Curve

## 1. Overview

The Precision-Recall curve shows the tradeoff between precision and recall across thresholds. It is especially useful for imbalanced classification where the positive class is rare.

Real-world usage:

- Fraud detection.
- Disease detection.
- Search and retrieval.
- Object detection.
- RAG context retrieval evaluation.

## 2. Intuition

As you lower the classification threshold, the model predicts more positives.

This usually:

- Increases recall.
- Decreases precision.

The PR curve shows how much precision you keep while increasing recall.

## 3. Prerequisites

- Precision
- Recall
- Probability scores
- Classification threshold
- Class imbalance
- Confusion matrix

## 4. Core Concepts

| Subtopic        | Meaning                   | Why it matters              | Simple example              | Interview angle                 |
|-----------------|---------------------------|-----------------------------|-----------------------------|---------------------------------|
| Precision       | $TP / (TP + FP)$          | Y-axis                      | Alert correctness           | "Why precision in PR curve?"    |
| Recall          | $TP / (TP + FN)$          | X-axis                      | Positive coverage           | "Why recall in PR curve?"       |
| Threshold sweep | Try many cutoffs          | Produces the curve          | 0.9 to 0.1                  | "Why curve changes?"            |
| Baseline        | Positive class prevalence | Random classifier reference | 1% fraud baseline precision | "Why PR is good for imbalance?" |

## 5. Algorithm / Working Process

1. Get true binary labels.
2. Get positive-class scores.
3. Sort thresholds.
4. For each threshold:
  - Predict positives.
  - Compute precision.
  - Compute recall.
5. Plot recall on x-axis and precision on y-axis.

Input:

- True labels
- Predicted scores

Output:

- Precision values, recall values, thresholds

## 6. Mathematical Foundation

```
Precision = TP / (TP + FP)
Recall = TP / (TP + FN)
```

PR curve:

```
PR = set of points (Recall(t), Precision(t)) for thresholds t
```

Baseline precision for random ranking:

```
Baseline = Number of positives / Total samples
```

Average precision is a common summary:

```
AP = sum_n (Recall_n - Recall_{n-1}) * Precision_n
```

## 7. Practical Implementation

```
import numpy as np
from sklearn.metrics import precision_recall_curve, average_precision_score
import matplotlib.pyplot as plt

y_true = np.array([0, 0, 1, 1, 0, 1, 0, 0])
y_score = np.array([0.05, 0.20, 0.40, 0.90, 0.10, 0.70, 0.30, 0.15])

precision, recall, thresholds = precision_recall_curve(y_true, y_score)
ap = average_precision_score(y_true, y_score)

plt.plot(recall, precision, label=f"AP = {ap:.2f}")
plt.xlabel("Recall")
plt.ylabel("Precision")
plt.title("Precision-Recall Curve")
plt.legend()
plt.show()
```

## 8. Code Explanation

- `precision_recall_curve` sweeps thresholds.
- `precision` and `recall` define curve points.
- `average_precision_score` summarizes the PR curve.
- PR curves use predicted scores, not only hard labels.

## 9. Training / Evaluation

Use PR curves when:

- Positives are rare.
- False positives matter.
- You care about the positive class more than negatives.

Deployment workflow:

1. Train model with log loss or another suitable loss.
2. Plot PR curve on validation data.
3. Choose an operating threshold based on required precision or recall.
4. Confirm selected threshold on test data.

## 10. Complexity and Cost

| Cost type     | Complexity                   |
|---------------|------------------------------|
| Time          | $O(n \log n)$ due to sorting |
| Memory        | $O(n)$                       |
| CPU/GPU       | CPU enough                   |
| Training cost | None                         |

## 11. Common Use Cases

- Rare disease detection
- Fraud detection
- Search result ranking
- RAG retrieval evaluation
- Recommendation systems
- Object detection evaluation

## 12. Common Mistakes

- Using PR curve with hard labels instead of scores.
- Comparing AP across datasets with different prevalence without context.
- Ignoring threshold selection.
- Assuming PR-AUC and ROC-AUC always agree.
- Not checking the required operating point.
- Forgetting that baseline precision equals positive rate.

## 13. Edge Cases / Limitations

PR curves are sensitive to class prevalence. If the positive rate changes between validation and production, precision may change.

PR curves can also be jagged on small datasets.

They focus on the positive class and do not directly show true negative behavior.

## 14. Variations

| Variation            | What changes                 | When to use            | Placement importance |
|----------------------|------------------------------|------------------------|----------------------|
| Average precision    | Weighted summary of PR curve | Ranking/reporting      | Very high            |
| PR-AUC trapezoidal   | Area under PR curve          | Approximate comparison | Medium               |
| Precision@k          | Precision at fixed top k     | Search/recommendation  | High                 |
| Recall@k             | Recall at fixed top k        | Retrieval/RAG          | High                 |
| Class-wise PR curves | One PR curve per class       | Multiclass/multilabel  | Medium               |

## 15. Related Topics

- Precision: y-axis.
- Recall: x-axis.
- F1-score: one threshold-specific balance point.
- ROC curve: uses TPR and FPR.
- AUC: scalar area summary.
- Average precision: common PR summary.

## 16. Interview Questions

1. What does PR curve plot? Precision against recall across thresholds.
2. When is PR curve better than ROC? When positive class is rare.
3. What is baseline precision? Positive class prevalence.
4. What happens when threshold decreases? Recall usually increases and precision may decrease.
5. What is average precision? A summary of precision values weighted by recall increase.
6. Does PR curve use true negatives directly? No.
7. Why is PR useful for fraud detection? It focuses on positive alerts and missed fraud.
8. Can PR curve be unstable? Yes, on small or highly noisy datasets.
9. What is precision@k? Fraction of relevant items among top k predictions.

10. Difference between PR-AUC and ROC-AUC? PR-AUC focuses on positive class quality; ROC-AUC includes FPR over negatives.

## 17. Practice Tasks

- Plot PR curve for an imbalanced dataset.
- Compute average precision.
- Choose threshold for precision above 90%.
- Compare PR and ROC curves.
- Implement precision@k and recall@k.

## 18. Project Ideas

| Project                 | What it does                           | Tech stack          | Dataset                  | Resume value             |
|-------------------------|----------------------------------------|---------------------|--------------------------|--------------------------|
| Fraud PR Dashboard      | Finds precision-recall operating point | Streamlit, sklearn  | Credit card fraud        | Strong applied ML        |
| RAG Retrieval Evaluator | Measures recall@k and precision@k      | Python, FAISS       | Natural Questions sample | AI engineering relevance |
| Rare Disease PR Study   | Compares PR-AUC across models          | sklearn, matplotlib | Medical dataset          | Metric depth             |

## 19. Quick Revision

- Key idea: Precision vs recall across thresholds.
- Main formula:  $\text{Precision} = \text{TP} / (\text{TP} + \text{FP})$ ,  $\text{Recall} = \text{TP} / (\text{TP} + \text{FN})$ .
- When to use: Imbalanced positive-class problems.
- Important metrics: AP, PR-AUC, F1, precision@k.
- Common traps: Comparing across different prevalences blindly.
- Interview one-liner: PR curve shows how much precision you keep as you recover more positives.

## 20. Final Cheat Sheet

| Item                | Summary                                   |
|---------------------|-------------------------------------------|
| Definition          | Curve of precision vs recall              |
| Input/output        | Labels and scores in, curve out           |
| Main steps          | Sweep threshold, compute precision/recall |
| Key hyperparameters | Positive class, threshold                 |
| Metrics             | AP, PR-AUC, precision@k, recall@k         |
| Pros                | Great for rare positives                  |
| Cons                | Sensitive to class prevalence             |
| Best use cases      | Fraud, retrieval, medical screening       |

# Log Loss

## 1. Overview

Log loss, also called cross-entropy loss, evaluates predicted probabilities. It penalizes confident wrong predictions heavily.

Unlike accuracy or F1, log loss cares not only whether the predicted class is correct, but how confident the model was.

Real-world usage:

- Logistic regression training.
- Neural network classification.
- Probability-based risk scoring.
- Kaggle competitions.
- Calibration-sensitive systems.

## 2. Intuition

If the true label is positive:

- Predicting 0.99 probability gives very low loss.
- Predicting 0.60 gives moderate loss.
- Predicting 0.01 gives huge loss.

Analogy: Log loss rewards honest confidence. Being wrong is bad; being confidently wrong is much worse.

## 3. Prerequisites

- Probability
- Logarithms
- Maximum likelihood estimation
- Binary and multiclass classification
- Softmax and sigmoid
- Cross-entropy

## 4. Core Concepts

| Subtopic              | Meaning                                                   | Why it matters                   | Simple example     | Interview angle              |
|-----------------------|-----------------------------------------------------------|----------------------------------|--------------------|------------------------------|
| Predicted probability | Confidence assigned to class                              | Log loss evaluates probabilities | P(spam)=0.9        | "How differs from accuracy?" |
| Cross-entropy         | Distance from true distribution to predicted distribution | Standard classification loss     | one-hot vs softmax | "Why used in neural nets?"   |

| Subtopic        | Meaning                                | Why it matters                 | Simple example        | Interview angle                             |
|-----------------|----------------------------------------|--------------------------------|-----------------------|---------------------------------------------|
| Confident error | Wrong prediction with high confidence  | Heavily penalized              | true=1, pred=0.01     | "Why log loss can explode?"                 |
| Calibration     | Probability matches observed frequency | Important for decision systems | 0.8 means 80% correct | "Does low log loss imply good calibration?" |

## 5. Algorithm / Working Process

Binary classification:

1. Model predicts probability  $p$  for positive class.
2. If true label is 1, use  $-\log(p)$ .
3. If true label is 0, use  $-\log(1-p)$ .
4. Average across samples.

Multiclass classification:

1. Model predicts probability distribution over classes.
2. Select probability assigned to true class.
3. Take negative log.
4. Average over samples.

## 6. Mathematical Foundation

Binary log loss:

$$\text{LogLoss} = -1/N * \sum_i [y_i \log(p_i) + (1 - y_i) \log(1 - p_i)]$$

where:

- $y_i$  is 0 or 1.
- $p_i$  is predicted probability for class 1.

Multiclass log loss:

$$\text{LogLoss} = -1/N * \sum_i \sum_c y_{\{i,c\}} \log(p_{\{i,c\}})$$

For one-hot labels, this becomes:

$$\text{LogLoss} = -1/N * \sum_i \log(\text{probability assigned to true class})$$

Optimization idea:

- Minimizing log loss is equivalent to maximizing likelihood of observed labels.

## 7. Practical Implementation

```
import numpy as np
from sklearn.metrics import log_loss

y_true = np.array([1, 0, 1, 1])
y_prob = np.array([0.90, 0.20, 0.60, 0.05])

eps = 1e-15
y_prob_clipped = np.clip(y_prob, eps, 1 - eps)

manual_loss = -np.mean(
    y_true * np.log(y_prob_clipped)
    + (1 - y_true) * np.log(1 - y_prob_clipped)
)

print("Manual log loss:", manual_loss)
print("sklearn log loss:", log_loss(y_true, y_prob))
```

## 8. Code Explanation

- `y_prob` stores positive-class probabilities.
- `np.clip` prevents `log(0)`.
- Correct confident predictions produce small loss.
- Confident wrong predictions produce large loss.
- `log_loss` is scikit-learn's implementation.

## 9. Training / Evaluation

Log loss is commonly used as the actual training objective.

Training:

- Logistic regression minimizes binary cross-entropy.
- Neural networks use sigmoid plus binary cross-entropy for binary tasks.
- Neural networks use softmax plus cross-entropy for multiclass tasks.

Evaluation:

- Lower log loss is better.
- Use validation log loss for early stopping.
- Compare with accuracy/F1 because low log loss does not always mean best thresholded decisions.

## 10. Complexity and Cost

| Cost type     | Complexity                                       |
|---------------|--------------------------------------------------|
| Time          | $O(n)$ for binary, $O(nk)$ for multiclass        |
| Memory        | $O(1)$ streaming or $O(nk)$ stored probabilities |
| CPU/GPU       | CPU for metric; GPU during neural training       |
| Training cost | Depends on model, not metric                     |

## 11. Common Use Cases

- Logistic regression
- Neural network classification
- Risk prediction
- Probabilistic classification
- Model calibration assessment
- Competition metrics

## 12. Common Mistakes

- Passing hard labels instead of probabilities.
- Forgetting to clip probabilities before manual log.
- Thinking accuracy and log loss always agree.
- Ignoring class imbalance.
- Reporting log loss without calibration analysis.
- Comparing log loss across datasets with different label noise.

## 13. Edge Cases / Limitations

Log loss is very sensitive to label noise. If labels are wrong, confident correct-looking predictions may be punished.

It can also reward calibrated probabilities even when top-1 accuracy is not best.

If a model predicts exactly 0 probability for the true class, loss becomes infinite.

## 14. Variations

| Variation                        | What changes              | When to use               | Placement importance |
|----------------------------------|---------------------------|---------------------------|----------------------|
| Binary cross-entropy             | Binary labels             | Binary classification     | Very high            |
| Categorical cross-entropy        | One-hot multiclass labels | Multiclass neural nets    | Very high            |
| Sparse categorical cross-entropy | Integer class labels      | Multiclass deep learning  | High                 |
| Weighted log loss                | Class weights added       | Imbalanced classification | High                 |

| Variation       | What changes              | When to use                  | Placement importance |
|-----------------|---------------------------|------------------------------|----------------------|
| Focal loss      | Downweights easy examples | Object detection/imbalance   | Medium-high          |
| Label smoothing | Softens one-hot labels    | Deep learning regularization | Medium               |

## 15. Related Topics

- Cross-entropy: general loss formulation.
- Softmax: converts logits to multiclass probabilities.
- Sigmoid: converts binary logits to probabilities.
- Calibration error: checks probability reliability.
- Brier score: another probability-quality metric.
- Accuracy: ignores probability confidence.

## 16. Interview Questions

1. What is log loss? A loss that evaluates predicted probabilities using negative log likelihood.
2. Binary formula?  $-[y \log(p) + (1-y) \log(1-p)]$  averaged over samples.
3. Why does log loss punish confident wrong predictions? Because  $\log(p)$  becomes very negative as probability of true class approaches zero.
4. Is lower log loss better? Yes.
5. Does log loss use probabilities or labels? Probabilities.
6. Why clip probabilities? To avoid  $\log(0)$ .
7. What is multiclass log loss? Negative log probability assigned to the true class, averaged.
8. Relation to maximum likelihood? Minimizing log loss maximizes likelihood of observed labels.
9. Can accuracy improve while log loss worsens? Yes, if predictions become more confidently wrong on some examples.
10. What is focal loss? A modified cross-entropy that focuses training on hard examples.

## 17. Practice Tasks

- Implement binary log loss manually.
- Compare accuracy and log loss for two probability models.
- Train logistic regression and monitor validation log loss.
- Show effect of confident wrong predictions.
- Implement weighted log loss for imbalanced data.

## 18. Project Ideas

| Project                            | What it does                         | Tech stack         | Dataset             | Resume value               |
|------------------------------------|--------------------------------------|--------------------|---------------------|----------------------------|
| Probability Quality Lab            | Compares accuracy vs log loss        | sklearn, NumPy     | Synthetic data      | Deep metric intuition      |
| Neural Classifier Loss Study       | Tracks cross-entropy during training | PyTorch            | MNIST/CIFAR-10      | Deep learning fundamentals |
| Calibration and Log Loss Dashboard | Studies probability reliability      | sklearn, Streamlit | Credit risk dataset | Production ML relevance    |

## 19. Quick Revision

- Key idea: Penalizes wrong probability confidence.
- Main formula:  $-[y \log p + (1-y) \log(1-p)]$ .
- When to use: Probabilistic classification.
- Important metrics: Log loss, calibration error, Brier score.
- Common traps: Passing labels instead of probabilities.
- Interview one-liner: Log loss asks whether the model assigned high probability to the true class.

## 20. Final Cheat Sheet

| Item                | Summary                                                  |
|---------------------|----------------------------------------------------------|
| Definition          | Negative log likelihood of true labels                   |
| Input/output        | True labels and probabilities in, loss out               |
| Main steps          | Take negative log probability of true class              |
| Key hyperparameters | Class weights, label smoothing if training               |
| Metrics             | Log loss, cross-entropy                                  |
| Pros                | Good for probability quality and training                |
| Cons                | Sensitive to label noise and overconfidence              |
| Best use cases      | Logistic regression, neural classification, risk scoring |

## Top-k Accuracy

### 1. Overview

Top-k accuracy checks whether the true label appears among the model's top  $k$  predicted classes. It is common in multiclass deep learning tasks with many possible labels.

It is useful when:

- Multiple classes are visually or semantically similar.
- The model returns a ranked list.
- Users inspect several suggestions.

Real-world usage:

- ImageNet classification.
- Product category prediction.
- Autocomplete systems.
- Medical decision support with candidate diagnoses.

## 2. Intuition

Top-1 accuracy asks: "Was the first answer correct?"

Top-5 accuracy asks: "Was the correct answer somewhere in the first five guesses?"

Example:

For an image of a golden retriever, the top predictions may be:

1. Labrador
2. Golden retriever
3. Flat-coated retriever

Top-1 is wrong, but top-3 is correct.

## 3. Prerequisites

- Multiclass classification
- Probability/logit scores
- Sorting/ranking
- Softmax outputs
- Accuracy

## 4. Core Concepts

| Subtopic             | Meaning                        | Why it matters           | Simple example               | Interview angle                      |
|----------------------|--------------------------------|--------------------------|------------------------------|--------------------------------------|
| Top-k predictions    | k highest scoring classes      | Candidate set            | top 5 ImageNet labels        | "What is top-k?"                     |
| True label inclusion | True class appears in top k    | Defines correctness      | true class rank 3            | "How is it different from accuracy?" |
| Choice of k          | Number of allowed guesses      | Controls leniency        | k=1 vs k=5                   | "Why top-5 in ImageNet?"             |
| Ranking quality      | Correct class should rank high | Useful beyond hard label | product category suggestions | "Where top-k matters?"               |

## 5. Algorithm / Working Process

1. For each sample, get class scores.
2. Sort class scores in descending order.
3. Select top **k** class indices.

4. Check whether the true label is in those indices.
5. Average correctness over all samples.

Input:

- True class labels
- Predicted class scores/probabilities/logits
- Value of  $k$

Output:

- Top-k accuracy score

## 6. Mathematical Foundation

For sample  $i$ , let  $\text{TopK}(x_i)$  be the set of top  $k$  predicted labels.

$$\text{Top-k Accuracy} = (1/N) * \sum_i I(y_i \text{ in TopK}(x_i))$$

where:

- $I(\dots)$  is 1 if condition is true, else 0.
- $N$  is number of samples.

Top-1 accuracy is standard accuracy.

## 7. Practical Implementation

```
import numpy as np
from sklearn.metrics import top_k_accuracy_score

y_true = np.array([2, 0, 1])
y_score = np.array([
    [0.10, 0.20, 0.70],
    [0.40, 0.35, 0.25],
    [0.50, 0.30, 0.20],
])

manual_top2 = np.mean([
    y_true[i] in np.argsort(y_score[i])[-2:]
    for i in range(len(y_true))
])

sklearn_top2 = top_k_accuracy_score(y_true, y_score, k=2, labels=[0, 1, 2])

print("Manual top-2 accuracy:", manual_top2)
print("sklearn top-2 accuracy:", sklearn_top2)
```

## 8. Code Explanation

- `y_score` contains class scores for each sample.
- `np.argsort(...)[-2:]` selects indices of two largest scores.
- The true label is checked against the top-2 indices.
- `top_k_accuracy_score` provides the standard implementation.

## 9. Training / Evaluation

Top-k accuracy is usually an evaluation metric, not a training loss.

Use it when:

- Model outputs ranked candidates.
- User can choose from suggestions.
- Top-1 accuracy is too strict for semantically close classes.

For training, use cross-entropy, then evaluate top-k accuracy.

## 10. Complexity and Cost

For `n` samples and `c` classes:

| Cost type          | Complexity                                           |
|--------------------|------------------------------------------------------|
| Full sort time     | $O(n * c \log c)$                                    |
| Partial top-k time | $O(n * c \log k)$ or better                          |
| Memory             | $O(nc)$ if storing all scores                        |
| CPU/GPU            | CPU enough for metric; GPU for large model inference |

## 11. Common Use Cases

- Image classification
- Product taxonomy prediction
- Search/autocomplete
- Recommendation candidate generation
- Medical diagnosis shortlist
- Large-vocabulary NLP classification

## 12. Common Mistakes

- Reporting only top-5 when top-1 matters in deployment.
- Using top-k for binary classification unnecessarily.
- Comparing different values of `k`.
- Forgetting to pass all labels to scikit-learn.
- Using probabilities after filtering classes incorrectly.
- Ignoring ties in scores.

## 13. Edge Cases / Limitations

Top-k accuracy becomes less meaningful when  $k$  is large relative to number of classes.

If there are 10 classes and  $k=9$ , top-k accuracy will likely look artificially high.

It also does not evaluate ranking order inside the top-k set.

## 14. Variations

| Variation            | What changes                | When to use               | Placement importance |
|----------------------|-----------------------------|---------------------------|----------------------|
| Top-1 accuracy       | $k=1$                       | Standard classification   | Very high            |
| Top-5 accuracy       | $k=5$                       | ImageNet-style evaluation | High                 |
| Recall@k             | Relevant item in top k      | Retrieval/RAG             | High                 |
| Mean reciprocal rank | Rewards higher rank         | Search/QA                 | Medium               |
| Hit rate@k           | Same idea in recommendation | Recommender systems       | Medium               |

## 15. Related Topics

- Accuracy: top-1 accuracy.
- Softmax: generates class probabilities.
- Cross-entropy: common training loss.
- Ranking metrics: MRR, NDCG, recall@k.
- PR curve: threshold-based rather than fixed-rank evaluation.

## 16. Interview Questions

1. What is top-k accuracy? It is correct if the true label appears in the top k predictions.
2. What is top-1 accuracy? Standard accuracy.
3. Why use top-5 accuracy? Useful when many classes are similar and shortlist quality matters.
4. Does top-k accuracy use probabilities? It uses scores or probabilities to rank classes.
5. Is top-k useful for binary classification? Usually no.
6. What happens as k increases? Top-k accuracy usually increases.
7. Does it care about exact rank within top k? No.
8. Which metric cares about rank position? MRR or NDCG.
9. Where is top-k common? ImageNet, recommendation, retrieval, autocomplete.
10. Why can top-k be misleading? Large k can make weak models look good.

## 17. Practice Tasks

- Implement top-k accuracy from scratch.
- Compare top-1, top-3, and top-5 accuracy.
- Evaluate a CNN on CIFAR-10 using top-k.
- Create ranking errors where top-1 fails but top-5 succeeds.
- Compare top-k accuracy with MRR.

## 18. Project Ideas

| Project                    | What it does                       | Tech stack           | Dataset                   | Resume value             |
|----------------------------|------------------------------------|----------------------|---------------------------|--------------------------|
| ImageNet-style Classifier  | Reports top-1 and top-5            | PyTorch, torchvision | CIFAR-100                 | Deep learning evaluation |
| Product Category Suggester | Returns top-3 categories           | sklearn/PyTorch      | E-commerce product titles | Practical ranking        |
| Diagnosis Shortlist Model  | Predicts top-k possible conditions | Python, sklearn      | Symptom dataset           | Decision-support framing |

## 19. Quick Revision

- Key idea: True label must be in top k predictions.
- Main formula: average of  $\mathbb{I}(y_i \text{ in TopK}_i)$ .
- When to use: Multiclass ranked outputs.
- Important metrics: Top-1, top-5, recall@k, MRR.
- Common traps: Choosing k too large.
- Interview one-liner: Top-k accuracy measures shortlist usefulness instead of only first-choice correctness.

## 20. Final Cheat Sheet

| Item                | Summary                                          |
|---------------------|--------------------------------------------------|
| Definition          | Correct if true class is in top k predictions    |
| Input/output        | Labels and class scores in, score out            |
| Main steps          | Rank classes, check true label inclusion         |
| Key hyperparameters | k                                                |
| Metrics             | Top-1, top-5, recall@k                           |
| Pros                | Useful for large multiclass shortlist tasks      |
| Cons                | Ignores exact rank inside top k                  |
| Best use cases      | Image classification, retrieval, recommendations |

## Matthews Correlation Coefficient

# 1. Overview

Matthews Correlation Coefficient, or MCC, is a classification metric that uses all four entries of the binary confusion matrix: TP, TN, FP, and FN.

It is especially useful for imbalanced binary classification because it remains informative even when class sizes differ strongly.

MCC ranges from -1 to 1:

- **1**: perfect prediction
- **0**: random-like prediction
- **-1**: perfectly wrong prediction

# 2. Intuition

MCC measures correlation between actual labels and predicted labels.

If predictions match labels strongly, MCC is high. If predictions are unrelated, MCC is near zero. If predictions are opposite, MCC is negative.

Analogy: MCC asks whether the model's decisions move in the same direction as the truth.

# 3. Prerequisites

- Confusion matrix
- Binary classification
- Correlation concept
- Class imbalance
- TP, TN, FP, FN

# 4. Core Concepts

| Subtopic              | Meaning                                     | Why it matters                    | Simple example                  | Interview angle    |
|-----------------------|---------------------------------------------|-----------------------------------|---------------------------------|--------------------|
| All confusion entries | TP, TN, FP, FN all used                     | More balanced metric              | Fraud and non-fraud both matter | "Why MCC robust?"  |
| Correlation           | Agreement between predicted and actual      | Gives -1 to 1 scale               | labels align                    | "Interpret MCC=0?" |
| Imbalance robustness  | Avoids majority-class inflation             | Useful for rare positives         | fraud dataset                   | "MCC vs accuracy?" |
| Symmetry              | Treats positive and negative classes fairly | Class label swap less problematic | disease vs healthy              | "Why use MCC?"     |

# 5. Algorithm / Working Process

1. Compute TP, TN, FP, FN.
2. Put them into the MCC formula.

3. Handle zero denominator if one class or prediction type is missing.
4. Interpret value from -1 to 1.

Input:

- True labels
- Predicted labels

Output:

- MCC score

## 6. Mathematical Foundation

$$\text{MCC} = \frac{(\text{TP} * \text{TN} - \text{FP} * \text{FN})}{\sqrt{(\text{TP} + \text{FP})(\text{TP} + \text{FN})(\text{TN} + \text{FP})(\text{TN} + \text{FN})}}$$

Interpretation:

- Numerator rewards correct positive and negative predictions.
- Numerator penalizes paired mistakes.
- Denominator normalizes by class and prediction counts.

If denominator is zero, MCC is undefined; libraries usually return 0.

## 7. Practical Implementation

```
import numpy as np
from sklearn.metrics import matthews_corrcoef

y_true = np.array([1, 0, 1, 0, 1, 0, 0, 0])
y_pred = np.array([1, 0, 0, 0, 1, 1, 0, 0])

mcc = matthews_corrcoef(y_true, y_pred)

print("MCC:", mcc)
```

## 8. Code Explanation

- `matthews_corrcoef` computes MCC from labels.
- It handles binary and multiclass cases.
- The result near 1 indicates strong agreement.
- A result near 0 indicates weak or random-like prediction.

## 9. Training / Evaluation

MCC is mainly an evaluation metric.

Use it when:

- Dataset is imbalanced.
- Both positive and negative classes matter.
- You need one robust metric.

Threshold tuning:

- Compute MCC across thresholds.
- Pick the threshold with highest validation MCC if MCC matches the business objective.

## 10. Complexity and Cost

| Cost type     | Complexity                        |
|---------------|-----------------------------------|
| Time          | $O(n)$                            |
| Memory        | $O(1)$ streaming or $O(n)$ stored |
| CPU/GPU       | CPU enough                        |
| Training cost | None                              |

## 11. Common Use Cases

- Bioinformatics classification
- Fraud detection
- Medical diagnosis
- Rare-event prediction
- Imbalanced binary classification
- Robust model comparison

## 12. Common Mistakes

- Not knowing how to interpret negative MCC.
- Reporting MCC without confusion matrix.
- Assuming MCC is only for balanced data.
- Ignoring threshold dependence.
- Comparing MCC across different positive class definitions.
- Forgetting denominator edge cases.

## 13. Edge Cases / Limitations

MCC can be undefined if predictions contain only one class or true labels contain only one class.

It is less intuitive for non-technical stakeholders than accuracy or recall.

MCC is not a probability metric and does not evaluate calibration.

## 14. Variations

| Variation       | What changes              | When to use           | Placement importance |
|-----------------|---------------------------|-----------------------|----------------------|
| Binary MCC      | Standard formula          | Binary classification | High                 |
| Multiclass MCC  | Generalized correlation   | Multiclass tasks      | Medium               |
| Thresholded MCC | MCC across thresholds     | Probability models    | Medium               |
| Class-wise MCC  | One-vs-rest MCC per class | Error analysis        | Low-medium           |

## 15. Related Topics

- Confusion matrix: MCC uses all entries.
- F1-score: ignores true negatives; MCC includes them.
- Cohen's kappa: chance-corrected agreement.
- Balanced accuracy: another imbalance-aware metric.
- Pearson correlation: MCC is equivalent to correlation for binary variables.

## 16. Interview Questions

1. What is MCC? A correlation-like classification metric using TP, TN, FP, and FN.
2. Range of MCC? From -1 to 1.
3. Interpret MCC = 1. Perfect prediction.
4. Interpret MCC = 0. No better than random-like agreement.
5. Interpret MCC = -1. Perfectly wrong prediction.
6. Why is MCC good for imbalanced data? It uses all confusion matrix entries and balances class effects.
7. Formula?  $(TP * TN - FP * FN) / \sqrt{(TP + FP)(TP + FN)(TN + FP)(TN + FN)}$ .
8. Does MCC use true negatives? Yes.
9. MCC vs F1? F1 ignores TN; MCC includes TN and is more balanced.
10. Is MCC differentiable? The metric on hard labels is not usually used directly as a training loss.

## 17. Practice Tasks

- Compute MCC manually from TP/TN/FP/FN.
- Compare MCC and accuracy on imbalanced data.
- Tune threshold for best MCC.
- Evaluate multiclass MCC.
- Create a case where F1 is decent but MCC is low.

## 18. Project Ideas

| Project                      | What it does               | Tech stack         | Dataset               | Resume value                  |
|------------------------------|----------------------------|--------------------|-----------------------|-------------------------------|
| Imbalanced Metric Comparator | Compares MCC, F1, accuracy | sklearn, Pandas    | Fraud dataset         | Strong placement discussion   |
| Bioinformatics Classifier    | Uses MCC as primary metric | Python, sklearn    | UCI molecular dataset | Research internship relevance |
| Threshold Selection Tool     | Finds best MCC threshold   | Streamlit, sklearn | Any binary dataset    | Deployment-aware metric skill |

## 19. Quick Revision

- Key idea: Correlation between true and predicted binary labels.
- Main formula: MCC confusion-matrix formula.
- When to use: Imbalanced classification where all outcomes matter.
- Important metrics: MCC, F1, balanced accuracy.
- Common traps: Harder interpretation and zero denominator.
- Interview one-liner: MCC is a balanced single-number metric that uses TP, TN, FP, and FN.

## 20. Final Cheat Sheet

| Item                | Summary                                       |
|---------------------|-----------------------------------------------|
| Definition          | Correlation between true and predicted labels |
| Input/output        | Labels in, score from -1 to 1 out             |
| Main steps          | Compute confusion entries and formula         |
| Key hyperparameters | Threshold                                     |
| Metrics             | MCC                                           |
| Pros                | Robust under imbalance                        |
| Cons                | Less intuitive than accuracy                  |
| Best use cases      | Rare-event and imbalanced classification      |

## Cohen's Kappa

### 1. Overview

Cohen's kappa measures agreement between two labelers or between a model and ground truth, corrected for agreement that could happen by chance.

In ML evaluation, it is useful when class imbalance makes raw accuracy misleading.

Real-world usage:

- Medical annotation agreement.

- Human labeler quality control.
- Model-vs-human agreement.
- NLP annotation tasks.
- Classification with skewed labels.

## 2. Intuition

Accuracy says: "How often did they agree?"

Cohen's kappa says: "How much better is the agreement than what we might expect by chance?"

Example:

If 95% of examples are negative, two annotators can agree often by always saying negative. Kappa corrects for this chance agreement.

## 3. Prerequisites

- Classification labels
- Confusion matrix
- Probability of agreement
- Class imbalance
- Annotation tasks

## 4. Core Concepts

| Subtopic           | Meaning                                      | Why it matters            | Simple example                     | Interview angle             |
|--------------------|----------------------------------------------|---------------------------|------------------------------------|-----------------------------|
| Observed agreement | Actual agreement rate                        | Similar to accuracy       | both say class A                   | "What is p <sub>o</sub> ?"  |
| Expected agreement | Agreement expected by chance                 | Corrects imbalance        | both often pick majority class     | "Why kappa not accuracy?"   |
| Chance correction  | Removes random agreement component           | More meaningful agreement | skewed labels                      | "Why useful in annotation?" |
| Weighted kappa     | Penalizes near/far disagreements differently | Ordinal classes           | rating 4 vs 5 less bad than 1 vs 5 | "When weighted kappa?"      |

## 5. Algorithm / Working Process

1. Build confusion matrix between two raters or true/predicted labels.
2. Compute observed agreement  $p_o$ .
3. Compute expected agreement  $p_e$  from marginal label frequencies.
4. Apply kappa formula.
5. Interpret agreement level.

Input:

- Labels from rater/model A

- Labels from rater/model B

Output:

- Kappa score

## 6. Mathematical Foundation

$$\text{kappa} = (p_o - p_e) / (1 - p_e)$$

where:

- $p_o$  = observed agreement
- $p_e$  = expected agreement by chance

For class  $c$ :

$$p_e = \sum_c P_A(c) * P_B(c)$$

Interpretation:

- 1: perfect agreement
- 0: agreement equal to chance
- < 0: worse than chance

## 7. Practical Implementation

```
import numpy as np
from sklearn.metrics import cohen_kappa_score

human_labels = np.array(["positive", "negative", "neutral", "positive",
                          "negative"])
model_labels = np.array(["positive", "negative", "positive", "positive",
                          "negative"])

kappa = cohen_kappa_score(human_labels, model_labels)

print("Cohen's kappa:", kappa)
```

## 8. Code Explanation

- `human_labels` represent one rater or ground truth.
- `model_labels` represent another rater or model.
- `cohen_kappa_score` calculates chance-corrected agreement.
- The result is more informative than accuracy when class frequencies are skewed.

## 9. Training / Evaluation

Cohen's kappa is usually used in evaluation and annotation analysis.

Use it to:

- Measure agreement between annotators.
- Check label quality before training.
- Compare model predictions with human labels.
- Evaluate ordinal classification with weighted kappa.

It is not commonly used as a neural network training loss.

## 10. Complexity and Cost

| Cost type     | Complexity                    |
|---------------|-------------------------------|
| Time          | $O(n)$                        |
| Memory        | $O(k^2)$ for confusion matrix |
| CPU/GPU       | CPU enough                    |
| Training cost | None                          |

## 11. Common Use Cases

- Medical diagnosis labeling
- Sentiment annotation
- Essay scoring
- Radiology report labeling
- Human evaluation of LLM outputs
- Model-human agreement studies

## 12. Common Mistakes

- Treating kappa like ordinary accuracy.
- Ignoring class prevalence effects.
- Using unweighted kappa for ordinal labels.
- Comparing kappa across very different datasets.
- Overinterpreting informal agreement scales.
- Forgetting kappa is for agreement, not probability quality.

## 13. Edge Cases / Limitations

Kappa can behave unexpectedly when label prevalence is extremely skewed. This is called the kappa paradox: high observed agreement can still produce a modest kappa.

It also assumes independent raters and may not reflect real annotation workflows where raters influence each other.

## 14. Variations

| Variation                | What changes                | When to use                      | Placement importance |
|--------------------------|-----------------------------|----------------------------------|----------------------|
| Cohen's kappa            | Two raters                  | Model vs human or two annotators | High                 |
| Weighted kappa           | Weighted disagreement       | Ordinal labels                   | High                 |
| Fleiss' kappa            | More than two raters        | Multi-annotator labeling         | Medium               |
| Quadratic weighted kappa | Squared penalty by distance | Ratings/grades                   | Medium               |

## 15. Related Topics

- Accuracy: observed agreement without chance correction.
- Confusion matrix: used to compute agreement.
- MCC: another balanced agreement-like metric.
- Inter-annotator agreement: broader labeling reliability topic.
- Human evaluation: important for NLP and LLM systems.

## 16. Interview Questions

1. What is Cohen's kappa? A chance-corrected agreement metric for two raters.
2. Formula?  $(p_o - p_e) / (1 - p_e)$ .
3. What is  $p_o$ ? Observed agreement.
4. What is  $p_e$ ? Expected agreement by chance.
5. Why use kappa instead of accuracy? It accounts for chance agreement, especially with imbalanced labels.
6. Range of kappa? Usually from -1 to 1.
7. What does kappa = 0 mean? Agreement is no better than chance.
8. When use weighted kappa? For ordinal labels where some mistakes are closer than others.
9. Can high accuracy have low kappa? Yes, especially with skewed class distributions.
10. What is Fleiss' kappa? A kappa variant for more than two raters.

## 17. Practice Tasks

- Compute Cohen's kappa for two annotators.
- Compare accuracy and kappa on imbalanced labels.
- Use weighted kappa for ordinal ratings.
- Simulate chance agreement and observe kappa.
- Analyze human-labeler quality before training.

## 18. Project Ideas

| Project                       | What it does                   | Tech stack                  | Dataset             | Resume value                |
|-------------------------------|--------------------------------|-----------------------------|---------------------|-----------------------------|
| Annotation Agreement Analyzer | Computes kappa among labelers  | Python, Pandas              | Custom labeling CSV | Great NLP/data role project |
| Essay Scoring Evaluator       | Uses weighted kappa for grades | sklearn, Pandas             | ASAP essay scoring  | Ordinal evaluation skill    |
| LLM Judge Agreement Study     | Compares human and LLM labels  | Python, OpenAI API optional | Human eval dataset  | AI evaluation relevance     |

## 19. Quick Revision

- Key idea: Agreement corrected for chance.
- Main formula:  $(p_o - p_e) / (1 - p_e)$ .
- When to use: Annotation agreement and skewed labels.
- Important metrics: Kappa, weighted kappa, accuracy.
- Common traps: Kappa paradox and wrong weighting.
- Interview one-liner: Cohen's kappa tells whether agreement is better than chance.

## 20. Final Cheat Sheet

| Item                | Summary                                     |
|---------------------|---------------------------------------------|
| Definition          | Chance-corrected agreement metric           |
| Input/output        | Two label sequences in, kappa out           |
| Main steps          | Compute observed and expected agreement     |
| Key hyperparameters | Weighting scheme                            |
| Metrics             | Cohen's kappa, weighted kappa               |
| Pros                | Handles chance agreement                    |
| Cons                | Can be unintuitive under extreme prevalence |
| Best use cases      | Annotation quality, model-human agreement   |

## Calibration Error

### 1. Overview

Calibration error measures whether predicted probabilities match real-world frequencies.

A classifier is calibrated if, among all examples where it predicts 80% probability, about 80% are actually positive.

Calibration matters when probabilities drive decisions.

Real-world usage:

- Medical risk prediction.
- Credit default probability.
- Weather-like event forecasting.
- Fraud risk ranking with manual review.
- LLM confidence estimation and selective prediction.

2. Intuition

Accuracy asks whether the model is right.

Calibration asks whether the model knows how confident it should be.

Example:

Suppose a model predicts 0.9 probability for 100 samples. If only 60 are correct, the model is overconfident.

Analogy: A student who says "I am 90% sure" should be right about 90% of the time. If they are right only 60% of the time, their confidence is poorly calibrated.

3. Prerequisites

- Probability estimates
- Binary or multiclass classification
- Binning
- Expected value
- Reliability diagrams
- Log loss and Brier score basics

4. Core Concepts

| Subtopic            | Meaning                                        | Why it matters                 | Simple example               | Interview angle                  |
|---------------------|------------------------------------------------|--------------------------------|------------------------------|----------------------------------|
| Confidence          | Model's predicted probability for chosen class | Basis of calibration           | max softmax = 0.8            | "What is confidence?"            |
| Accuracy per bin    | Actual correctness in probability interval     | Compares confidence to reality | bin 0.8-0.9 has 75% accuracy | "How ECE is computed?"           |
| Reliability diagram | Plot confidence vs accuracy                    | Visual calibration check       | diagonal is calibrated       | "How diagnose calibration?"      |
| Overconfidence      | Confidence > actual accuracy                   | Common in deep nets            | 95% confident, 70% right     | "Why neural nets miscalibrated?" |
| Calibration method  | Adjust probabilities after training            | Improves decision quality      | temperature scaling          | "How fix calibration?"           |

5. Algorithm / Working Process

Expected Calibration Error, or ECE:

1. Get predicted probabilities.
2. For each sample, compute confidence as maximum predicted probability.
3. Compute correctness: whether predicted class equals true class.
4. Split predictions into confidence bins.
5. For each bin:
  - Compute average confidence.
  - Compute average accuracy.
  - Take absolute difference.
6. Weight each bin difference by number of samples.
7. Sum weighted differences.

Input:

- True labels
- Predicted probabilities
- Number of bins

Output:

- Calibration error score, lower is better

## 6. Mathematical Foundation

For bin  $B_m$ :

```
acc(B_m) = average correctness in bin  
conf(B_m) = average confidence in bin
```

Expected Calibration Error:

$$ECE = \sum_m (|B_m| / n) * |acc(B_m) - conf(B_m)|$$

Maximum Calibration Error:

$$MCE = \max_m |acc(B_m) - conf(B_m)|$$

Brier score for binary classification:

$$\text{Brier} = (1/N) * \sum_i (p_i - y_i)^2$$

Important note:

- ECE depends on binning choice.
- Log loss and Brier score are proper scoring rules.

## 7. Practical Implementation

```
import numpy as np

def expected_calibration_error(y_true, y_prob, n_bins=10):
    confidences = np.max(y_prob, axis=1)
    predictions = np.argmax(y_prob, axis=1)
    correct = predictions == y_true

    bin_edges = np.linspace(0.0, 1.0, n_bins + 1)
    ece = 0.0

    for lower, upper in zip(bin_edges[:-1], bin_edges[1:]):
        in_bin = (confidences > lower) & (confidences <= upper)
        prop_in_bin = np.mean(in_bin)

        if prop_in_bin == 0:
            continue

        accuracy_in_bin = np.mean(correct[in_bin])
        confidence_in_bin = np.mean(confidences[in_bin])
        ece += prop_in_bin * abs(accuracy_in_bin - confidence_in_bin)

    return ece

y_true = np.array([0, 1, 1, 0, 2])
y_prob = np.array([
    [0.80, 0.10, 0.10],
    [0.20, 0.70, 0.10],
    [0.10, 0.60, 0.30],
    [0.40, 0.50, 0.10],
    [0.20, 0.20, 0.60],
])

print("ECE:", expected_calibration_error(y_true, y_prob, n_bins=5))
```

## 8. Code Explanation

- `confidences` stores the model's highest class probability.
- `predictions` stores predicted class labels.
- `correct` records whether each prediction is right.
- Confidence values are grouped into bins.
- Each bin compares average confidence with average accuracy.
- ECE is the weighted average gap across bins.

## 9. Training / Evaluation

Calibration is evaluated after training.

Workflow:

1. Train classifier normally.
2. Evaluate accuracy, F1, log loss, and calibration on validation data.
3. If probabilities are overconfident, apply calibration on validation/calibration split.
4. Test calibrated model on held-out test data.

Common calibration methods:

- Platt scaling
- Isotonic regression
- Temperature scaling for neural networks

Avoid calibrating on the test set.

## 10. Complexity and Cost

| Cost type        | Complexity                                     |
|------------------|------------------------------------------------|
| ECE time         | $O(n + b)$                                     |
| Memory           | $O(n)$ for stored probabilities                |
| Training cost    | None for metric                                |
| Calibration cost | Low for Platt/temperature, higher for isotonic |
| CPU/GPU          | CPU enough for metric                          |

## 11. Common Use Cases

- Risk scoring
- Medical decision support
- Credit scoring
- Fraud probability estimation
- Uncertainty-aware ML
- Selective prediction
- Human-in-the-loop systems

## 12. Common Mistakes

- Assuming softmax probability is always reliable confidence.
- Evaluating calibration with too few samples per bin.
- Calibrating on test data.
- Improving calibration but ignoring accuracy.
- Reporting ECE without number of bins.
- Confusing calibration with discrimination/ranking.

## 13. Edge Cases / Limitations

ECE depends on bin choices. Different numbers of bins can produce different values.

Calibration can improve probability reliability but may not improve accuracy or AUC.

On small datasets, bin estimates are noisy.

Deep neural networks are often overconfident, especially under distribution shift.

## 14. Variations

| Variation           | What changes                          | When to use                    | Placement importance |
|---------------------|---------------------------------------|--------------------------------|----------------------|
| ECE                 | Average weighted calibration gap      | Common calibration reporting   | High                 |
| MCE                 | Worst bin gap                         | Safety-critical systems        | Medium               |
| Adaptive ECE        | Equal-frequency bins                  | Uneven confidence distribution | Medium               |
| Class-wise ECE      | Calibration per class                 | Multiclass debugging           | Medium               |
| Temperature scaling | Divides logits by learned temperature | Neural network calibration     | High                 |
| Isotonic regression | Non-parametric probability mapping    | Enough validation data         | Medium               |
| Platt scaling       | Logistic calibration                  | Binary classifiers             | Medium               |

## 15. Related Topics

- Log loss: rewards good probabilities.
- Brier score: squared probability error.
- Reliability diagram: visual calibration.
- ROC-AUC: ranking, not calibration.
- Temperature scaling: simple neural calibration method.
- Uncertainty estimation: broader confidence modeling.

## 16. Interview Questions

1. What is calibration? Agreement between predicted probabilities and observed frequencies.
2. What does it mean if a model predicts 0.8 confidence? Among similar 0.8 predictions, about 80% should be correct.
3. What is ECE? Expected Calibration Error, a binned confidence-accuracy gap.
4. Formula for ECE?  $\sum_m |B_m|/n * |acc(B_m) - conf(B_m)|$ .
5. Does high accuracy imply good calibration? No.
6. Does high AUC imply good calibration? No, AUC measures ranking.

7. Why are neural networks often overconfident? Cross-entropy training and overparameterization can produce high softmax confidence.
8. How can calibration be improved? Temperature scaling, Platt scaling, or isotonic regression.
9. Why not calibrate on test data? It causes data leakage and optimistic evaluation.
10. What is a reliability diagram? A plot comparing predicted confidence with empirical accuracy.

## 17. Practice Tasks

- Implement ECE from scratch.
- Plot reliability diagram.
- Compare accuracy, log loss, and ECE.
- Apply temperature scaling to a neural network.
- Study calibration under distribution shift.

## 18. Project Ideas

| Project                  | What it does                           | Tech stack        | Dataset                     | Resume value               |
|--------------------------|----------------------------------------|-------------------|-----------------------------|----------------------------|
| Calibration Dashboard    | Shows reliability diagrams and ECE     | Python, Streamlit | CIFAR/MNIST outputs         | Advanced evaluation skill  |
| Medical Risk Calibrator  | Calibrates disease probabilities       | sklearn           | Breast cancer/heart disease | Safety-aware ML            |
| LLM Confidence Evaluator | Measures answer confidence reliability | Python, Pandas    | QA logs                     | Modern AI evaluation angle |

## 19. Quick Revision

- Key idea: Probability confidence should match real correctness.
- Main formula:  $ECE = \sum |B_m|/n * |acc(B_m) - conf(B_m)|$ .
- When to use: Probability-driven decisions.
- Important metrics: ECE, MCE, Brier score, log loss.
- Common traps: Assuming softmax confidence is calibrated.
- Interview one-liner: Calibration checks whether 80% confidence really means 80% correctness.

## 20. Final Cheat Sheet

| Item                | Summary                                                   |
|---------------------|-----------------------------------------------------------|
| Definition          | Error between predicted confidence and empirical accuracy |
| Input/output        | Labels and probabilities in, calibration score out        |
| Main steps          | Bin confidences, compare bin accuracy and confidence      |
| Key hyperparameters | Number of bins, binning strategy                          |
| Metrics             | ECE, MCE, Brier score, log loss                           |

| Item           | Summary                                        |
|----------------|------------------------------------------------|
| Pros           | Crucial for risk-based decisions               |
| Cons           | Bin-sensitive and data-hungry                  |
| Best use cases | Medical, finance, safety, uncertainty-aware ML |